VIETNAM NATIONAL UNIVERSITY, HANOI

UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Bui Minh Thang

DESIGN AND EVALUATION OF A CONTEXT-AWARE RESOURCE PROFILE SELECTION EXTENSION FOR JUPYTERHUB ON KUBERNETES

GRADUATION THESIS

Major: Computer Networks and Data Communications

HA NOI – 2026

VIETNAM NATIONAL UNIVERSITY, HANOI

UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Bui Minh Thang

DESIGN AND EVALUATION OF A CONTEXT-AWARE RESOURCE PROFILE SELECTION EXTENSION FOR JUPYTERHUB ON KUBERNETES

GRADUATION THESIS

Major: Computer Networks and Data Communications

Supervisor: Dr. Hoang Xuan Tung

HA NOI – 2026

ABSTRACT

JupyterHub provides a flexible platform for interactive computing, but selecting an appropriate computational environment remains a manual configuration task. Users are typically required to understand available resource profiles, software environments, and notebook images before starting a session. As the number and diversity of available environments increase, manual selection can introduce unnecessary configuration effort and may result in unsuitable resource or software choices. This thesis investigates the use of workload intent and context to assist users in selecting an appropriate JupyterHub environment while preserving administrator-defined resource and deployment policies.

The problem addressed in this thesis is the selection of a suitable resource profile and notebook environment from a bounded, administrator-managed catalog based on a user's natural-language workload description. The objective is not to allow an automated model to generate arbitrary Kubernetes configurations, but to determine whether natural-language intent can be transformed into a structured representation, used to retrieve relevant approved environments, and subsequently filtered and ranked according to explicit technical constraints. A further objective is to establish a system architecture in which recommendation remains explainable, policy-bounded, reproducible, and subject to user confirmation before deployment.

The proposed approach, implemented as Intent-Spawner, combines structured intent extraction, hybrid information retrieval, deterministic constraint evaluation, and trusted catalog resolution. A user's workload description is converted into a schema-validated structured intent containing task characteristics, software requirements, resource constraints, and preferences. Candidate environments are then retrieved using two complementary retrieval channels: BM25-based sparse retrieval for exact technical terminology and dense semantic retrieval for semantically related workload descriptions. Their ranked results are combined using Reciprocal Rank Fusion (RRF). Retrieved candidates are subsequently evaluated against hard constraints, including GPU availability, minimum CPU and memory requirements, dataset size, and forbidden features. Candidates that violate hard constraints are excluded before deterministic preference ranking is applied.

The resulting recommendation is resolved through an administrator-owned candidate catalog rather than allowing the recommendation model to directly specify an image or Kubernetes

resource configuration. Approved notebook images are identified by immutable digest-pinned references, while resource profiles are restricted to administrator-defined configurations. A preview and confirmation mechanism binds the displayed recommendation to the authenticated user and to a single-use, time-limited token. The JupyterHub pre-spawn hook validates this state before applying the selected configuration, without performing new recommendation or model inference during pod creation. The system was evaluated through controlled experiment protocols covering recommendation behavior, system integration, resource-related measurements, and environment functionality. The evaluation distinguishes workload families as the semantic experimental unit and separates repeated executions used for stability analysis from independent recommendation cases.

The experimental evaluation examined recommendation quality and linguistic robustness on Benchmark v1 (471 test cases across 157 families with empirical Linux cgroups v2 ground truth), resource outcomes across 800 physical Kubernetes Pod trials and a 50-node cluster multi-tenancy simulation under Alibaba PAI GPU traces (Stream E4), and image capability through deterministic probes and OCI layer accounting for four approved images (Stream E5). P2 achieved 73.46% Candidate Accuracy@3 and 81.32% Profile Accuracy@3 (+47.13% gain over rule heuristics), while linguistic robustness evaluation demonstrated consistent stability across Canonical English, Informal/Noisy text, and localized Vietnamese queries ($|\Delta| \leq 1.28\%$). The evaluation also established the functional integration of recommendation preview, policy validation, user confirmation, and pre-spawn application within JupyterHub, including enforcement of approved profiles and digest-pinned images.

These results indicate that intent-aware environment selection can be integrated into JupyterHub while maintaining a separation between language-based recommendation and authoritative infrastructure configuration. Hybrid retrieval provides a mechanism for combining exact technical vocabulary with semantic similarity, whereas deterministic constraints prevent soft relevance signals from overriding explicit resource or policy requirements. The architecture therefore provides a bounded form of decision support in which natural-language interaction reduces configuration effort without transferring deployment authority to the recommendation model.

Keywords: JupyterHub, Kubernetes, Workload Intent, Resource Profile Recommendation, Hybrid Information Retrieval, Constraint-Based Selection, Notebook Environments.

AUTHORSHIP

I hereby declare that the work contained in this thesis is of my own and has not been previously submitted for a degree or diploma at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no materials previously published or written by another person except where due reference or acknowledgement is made.

Signature: _____

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my thesis supervisor, Dr. Hoang Xuan Tung, for guidance, critical feedback, and encouragement throughout the development and evaluation of this work. I am also grateful to the lecturers and staff of the Faculty of Information Technology, for the technical and academic foundation that made this project possible.

I would also like to thank my friends, and family for their valuable discussions, encouragement, and practical support throughout this iterative process. I take full responsibility for any remaining errors, limitations, or interpretation choices in this thesis.

Contents

Abstract i

Authorship ii

Acknowledgements iii

List of Figures ix

List of Tables xi

List of Algorithms xii

Abbreviations xiii

Chapter 1. Introduction 1

1.1. Problem statement 1

1.2. Contributions and thesis overview 3

Chapter 2. Background 5

2.1. JupyterHub 5

2.2. Kubernetes execution model 8

2.2.1. Containers and pods 8

2.2.2. CPU and memory resources 9

2.2.3. Resource profiles 10

2.3. Notebook container images 11

2.4. Workload intent 12

2.5. Natural language processing for intent extraction 13

2.5.1. Information extraction and slot-filling 14

2.5.2. Rule-based and finite-state methods 14

2.5.3. Dense neural representations and language models 15

2.6. Information retrieval 16
2.6.1. Sparse retrieval and BM25 17
2.6.2. Dense retrieval 18
2.6.3. Hybrid retrieval with reciprocal rank fusion 19
2.7. Constraint-based recommendation 20
Chapter 3. Proposed Method 22
3.1. Overview 22
3.2. Structured intent representation and local NLP extraction pipeline 24
3.2.1. Problem formulation: Structured information extraction and slot-filling 24
3.2.2. The local deterministic NLP extraction architecture 26
3.2.3. Computational efficiency and controller footprint guarantees 31
3.3. Hybrid candidate retrieval 32
3.3.1. Retrieval pipeline architecture 32
3.3.2. Sparse lexical and dense semantic retrieval channels 33
3.3.3. Rank fusion via Reciprocal Rank Fusion (RRF) 33
3.3.4. Architectural justification: Why hybrid retrieval and constraint filtering over direct LLM generation? 34
3.4. Constraint filtering 37
3.5. Multi-criteria candidate ranking 41
3.5.1. Scoring formulation 42
3.5.2. Utility and resource cost modeling 42
3.6. JupyterHub integration and runtime enforcement 43
3.6.1. Policy validation and candidate resolution 44
3.6.2. Two-phase preview and confirmation 44

3.6.3. Pre-spawn binding and execution 46
3.6.4. Policy-bounded dynamic resource overlay 47
Chapter 4. Evaluation 49
4.1. Experimental Framework and Testbed Setup 49
4.1.1. Evaluation Methodology and Multi-Stream Workflow 49
4.1.2. Input Workload Benchmark Taxonomy and Provenance 52
4.1.3. Testbed Hardware and Software Specifications 54
4.1.4. Formal Input and Output Artifact Definitions 56
4.1.5. Implementation, Reproducibility, and Evidence Boundaries 57
4.2. Offline recommendation performance 58
4.2.1. End-to-end recommendation accuracy 58
4.2.2. Retrieval channel ablation study 61
4.2.3. Robustness to query variations 63
4.2.4. Accuracy-cost trade-off analysis 64
4.3. Cluster resource profiling 65
4.3.1. Pod scheduling and runtime overhead 65
4.3.2. Container resource usage profiling 68
4.4. Storage Analysis, End-to-End Latency, and Cluster Simulation 70
4.4.1. Image capability probing 70
4.4.2. OCI layer and storage overhead 72
4.4.3. End-to-End Evaluation and Latency Breakdown 76
4.4.4. Large-Scale Cluster Trace Simulation 80
Chapter 5. Conclusion 87
5.1. Conclusion 87

5.2. Future work 88

References 88

List of Figures

2.1. Subsystem architecture of JupyterHub showing the Hub, Configurable HTTP Proxy, Spawner, and single-user notebook servers [15]……7

2.2. Conceptual structure of a Kubernetes Pod encapsulating containers and shared storage volumes [23]…… 9

2.3. Layered container image architecture with stacked read-only layers and a top read-write container layer [5] ……11

3.1. Overview of the proposed P2 intent-aware recommendation and execution architecture ……23

3.2. Hybrid candidate retrieval pipeline combining sparse lexical and dense semantic channels with Reciprocal Rank Fusion ……32

3.3. Deterministic hard constraint evaluation decision tree and pruning logic……40

3.4. Two-phase preview, user confirmation, and pre-spawn binding lifecycle ……45

4.1. Multi-stream experimental evaluation framework of Intent-Spawner, separating offline recommendation benchmarks, Kubernetes cluster trials, container capability sandboxes, and OCI manifest storage accounting.……52

4.2. Candidate retrieval channel ablation across Recall@1, Recall@3, and Recall@5, comparing Sparse BM25, Dense Semantic (all-MiniLM-L6-v2), and Hybrid RRF fusion. ……62

4.3. Observed Kubernetes Pod execution reliability across 800 real trials in Stream E4 (left), and the conceptual trade-off between OOM risk and resource savings across profile allocation strategies (right).……67

4.4. Locally observed memory envelopes (P2 Safe Envelope) compared against Static Small (256 MiB) and Static Large (2048 MiB) profiles across 20 workload families in the E4 experiment.……69

4.5. Three-tier container image verification architecture in Stream E5, separating gold-label correctness (Dimension A), catalog metadata soundness (Dimension B), and in-container runtime capability execution (Dimension C)..... 71

4.6. Cumulative compressed OCI layer storage across the four-image catalog prefix in Stream E5, demonstrating a 27.61% deduplication saving achieved through shared base layers. 74

4.7. Marginal unique compressed layer storage introduced by each container image in Stream E5, showing shared base layer reuse between general-purpose and deep learning images. 75

4.8. Pairwise shared layer storage heatmap (GiB) among the four approved container images, highlighting substantial layer inheritance between data science and deep learning environments. 76

4.9. Discrete-event simulation results on a large-scale cluster (1,000 vCPU, 100 GPU, 2,000 GB RAM across 50 nodes) driven by empirical E4 workload traces over an 8-hour diurnal cycle: (a) Memory allocation vs. physical consumption, (b) Pending notebook queue length, (c) Cluster capacity stranding percentage, and (d) Operational trade-off between OOM failure rate and multi-tenancy concurrency multiplier. 83

List of Tables

3.1. Specification of the StructuredIntent data contract and normalization rules 26

3.2. Deterministic hard constraint evaluation rules and pruning conditions 39

4.1. Multi-stream experimental evaluation matrix covering E1, E2, E4, and E5, evaluated pillars, inputs, testbeds, outputs, and empirical scope 51

4.2. Hardware and software specifications of the experimental testbeds 55

4.3. Empirical evidence boundaries and unsupported extrapolations across experiment streams 58

4.4. Observed E1 offline recommendation quality metrics on Benchmark v1 (471 test cases across 157 families) 60

4.5. Observed E2 semantic and linguistic robustness metrics across prompt variations on Benchmark v1 64

4.6. Observed E4 Kubernetes Pod execution outcomes across 800 trials (OrbStack environment) 67

4.7. E5 container image functional capability probe evaluation results 72

4.8. Observed compressed OCI layer-byte accounting and deduplication across four images 73

4.9. Marginal unique compressed layer bytes introduced by image order 74

4.10. End-to-end execution latency breakdown across pipeline stages (mean ± standard deviation or point estimates) 79

4.11. Large-scale cluster multi-tenancy simulation metrics across allocation strategies (1,000 vCPU, 100 GPU, 2,000 GB RAM, 800 Sessions) 84

List of Algorithms

3.1. Deterministic local NLP information extraction and contract validation procedure 30

3.2. Reciprocal Rank Fusion for hybrid candidate retrieval 34

3.3. Deterministic candidate evaluation, multi-criteria scoring, and tie-breaking 43

3.4. JupyterHub pre-spawn policy validation and deterministic resource binding 46

Abbreviations

API	Application Programming Interface
BM25	Best Matching 25
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
IR	Information Retrieval
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
OCI	Open Container Initiative
POD	Plain Old Data / Kubernetes Pod (context dependent)
RRF	Reciprocal Rank Fusion
RQ	Research Question
TTL	Time to Live
VPA	Vertical Pod Autoscaler

Chapter 1.

Introduction

1.1. Problem statement

The development of interactive computing and container orchestration has progressively changed how computational environments are provided to users. In 2014, the Jupyter Project branched off from the IPython project as a project aimed at supporting interactive data science and scientific computing across all programming languages. Around the same time that Jupyter was emerging, the open source project Kubernetes was introduced as a system to manage containerized workloads. Kubernetes received its first public commit in June 2014 and reached 1.0 in July 2015. JupyterHub, a multi-user version of the Jupyter notebook, was released as a preview in March 2015. The combination of JupyterHub and container orchestration technologies such as Kubernetes has since provided a practical foundation for delivering isolated and centrally managed notebook environments to groups of users. Today, JupyterHub explicitly supports Kubernetes-based deployments and is designed to provide users with computational environments and resources while allowing administrators to manage the underlying infrastructure.

Interactive computational environments have become an important part of modern data analysis, scientific computing, education, and machine learning. Jupyter notebooks are particularly useful in these settings because they allow users to combine executable code, computational results, and explanatory text within a single interactive environment. In multi-user deployments, however, providing notebook environments is not only a matter of running the notebook application itself. The platform must also manage users, compute resources, storage, software environments, and isolation between workloads. JupyterHub provides a common platform for deploying and managing multi-user Jupyter environments, while Kubernetes can provide the underlying infrastructure for scalable and isolated

execution. In a typical JupyterHub deployment on Kubernetes, KubeSpawner creates a Kubernetes Pod for each user's notebook server and can expose a set of administrator-defined resource profiles and container images. Such profiles allow administrators to control the available CPU, memory, accelerators, and software environments while preventing users from directly requesting arbitrary infrastructure configurations. This administrator-controlled approach provides an important operational boundary, but it also introduces a usability problem. Users generally describe their requirements in terms of the work they want to perform rather than in terms of infrastructure configuration. For example, a user may know that they need to train a machine-learning model on a large dataset, process images, or perform data analysis with a particular framework. The platform, however, may require the user to select an appropriate resource profile and notebook image from a predefined catalog. Consequently, users may need to understand technical differences between available environments before they can start their actual work. This creates a gap between workload intent and infrastructure configuration. A user understands the intended computation, while the platform requires a concrete profile and image selection. Bridging this gap is particularly relevant when a catalog contains multiple environments with different resource capacities and software capabilities.

One way to present users with the options of administrator-defined solution profiles is to create a simple list that users can select from, one at a time. But that way users will have to interpret the solution catalog for themselves. A simple name for a profile does not necessarily imply that the solution will be suitable for a given workload. And relying on familiar software names to pick environments does not necessarily guarantee that the chosen environment meets the user's explicit resource or capability requirements. Automating this decision is not straightforward either. A purely rule-based approach can provide deterministic and easily explainable decisions, but its ability to handle the variety of natural-language workload descriptions may be limited. Conversely, semantic retrieval or language-model-based approaches can improve the ability to interpret natural-language requests, but semantic relevance alone cannot guarantee that a candidate satisfies mandatory requirements or conforms to administrator-defined policies. The problem addressed by this thesis is therefore the design of a context-aware, policy-bounded

mechanism for recommending an appropriate JupyterHub resource profile and notebook image from an administrator-controlled catalog based on a user's workload intent. The problem is deliberately narrower than general cloud resource optimization. This system does not attempt to predict exactly how much resources a given workload will use to run, replace the scheduling provided by Kubernetes, or try to optimize the whole cluster. What this system tries to do is decide on which of the pre-authorized environments by an administrator a user can select from their descriptions of intended workloads, and lists all relevant environments that satisfy all given requirements and constraints of the platform.

1.2. Contributions and thesis overview

The main contributions of this thesis are twofold. First, the thesis develops a context-aware resource profile selection approach that connects natural-language workload descriptions with a finite catalog of administrator-approved JupyterHub environments. The approach considers workload intent, explicit resource requirements, and the available environment catalog as relevant context during the selection process. This allows users to describe their computational needs at the workload level while keeping the actual resource environments within the set of configurations defined and controlled by the platform administrator. Second, the thesis proposes a multi-dimensional evaluation framework for assessing the resource profile selection approach. Rather than relying on a single recommendation metric, the evaluation considers several aspects of system behavior, including recommendation quality, robustness to variations in workload descriptions, resource-related behavior, notebook-image functionality, and practical integration with the JupyterHub environment. This broader evaluation is intended to provide a more comprehensive view of the effectiveness and practical applicability of the proposed approach.

The remainder of the thesis is organized into four chapters. Chapter 2 presents the technical and conceptual foundations required for the proposed work. It introduces JupyterHub and Kubernetes-based notebook environments, resource profiles and container

images, workload intent, information retrieval, semantic retrieval, and recommendation-related concepts. These foundations establish the terminology and technical background used throughout the subsequent chapters. Chapter 3 presents the design of the proposed resource profile selection approach in detail. It describes the overall system architecture, workload-intent representation, candidate retrieval and ranking process, constraint handling, administrator-controlled environment catalog, and integration with the JupyterHub pre-spawn workflow. The chapter also discusses the main design decisions and defines the scope and boundaries of the proposed system. Chapter 4 describes the implementation and experimental methodology used to assess the proposed approach. It presents the experimental setup, evaluation scenarios, baselines, and evaluation metrics, followed by the experimental results and their discussion. The chapter also examines the limitations and scope of the evidence obtained from the experiments. Chapter 5 summarizes the work and its main findings, discusses the limitations of the current approach, and presents directions for future development and further evaluation. Overall, this thesis investigates the use of workload intent as an interface for selecting appropriate notebook environments in JupyterHub on Kubernetes. The central aim is to reduce the gap between user-level descriptions of computational tasks and infrastructure-level environment selection while maintaining administrator control and adherence to explicit platform constraints.

Chapter 2.

Background

This chapter introduces the technical concepts and methods required to understand the design of the proposed system. The discussion starts with interactive computational environments and their multi-user management through JupyterHub. It then describes Kubernetes-based execution, resource profiles, and notebook container images. The chapter subsequently introduces workload intent representation, natural language processing for information extraction, and the information retrieval techniques used for matching a natural-language request against a catalog of computational environments. Finally, constraint-based recommendation is discussed as the mechanism for ensuring that retrieved candidates satisfy explicit resource and environment requirements. The purpose of this chapter is to establish the concepts used later in the system design and evaluation. The implementation-specific architecture of the proposed system is presented separately in Chapter 3.

2.1. JupyterHub

Interactive computational environments are widely used for data analysis, scientific computing, machine learning, education, and software development. Instead of separating source code, execution, and results into independent tools, an interactive notebook allows these elements to be combined in a single computational document [19]. Jupyter Notebook is one of the most widely adopted environments following this model. At a basic level, a notebook server provides an interface through which a user can execute code interactively and inspect the resulting output. The Jupyter architecture separates the user-facing notebook interface from the kernel responsible for executing computations. Consequently, the computational environment can be configured independently of the browser-based interface. This separation is particularly useful in environments where

users have different computational requirements. A lightweight educational notebook may require only a basic Python environment, whereas a machine-learning workload may require additional libraries, larger amounts of memory, or specialized hardware accelerators. Therefore, providing an interactive notebook service at scale requires more than simply making the Jupyter interface available: the underlying execution environment must also be provisioned and isolated appropriately.

JupyterHub extends the Jupyter ecosystem from a single-user environment to a multi-user service [34]. It provides authentication, user management, and the creation of individual single-user notebook servers. The JupyterHub documentation describes the spawner as the component responsible for starting the single-user server and allocating its execution environment [15]. Conceptually, a JupyterHub deployment consists of several cooperating components, as illustrated in Figure 2.1. The Hub coordinates authentication through an Authenticator and manages user sessions recorded in a state database, while a Configurable HTTP Proxy routes incoming web requests either to the Hub for login operations or directly to the running notebook server of an authenticated user. A Spawner is responsible for creating and managing each user's computational environment upon demand. The resulting architecture separates service management from the actual execution of notebook workloads.

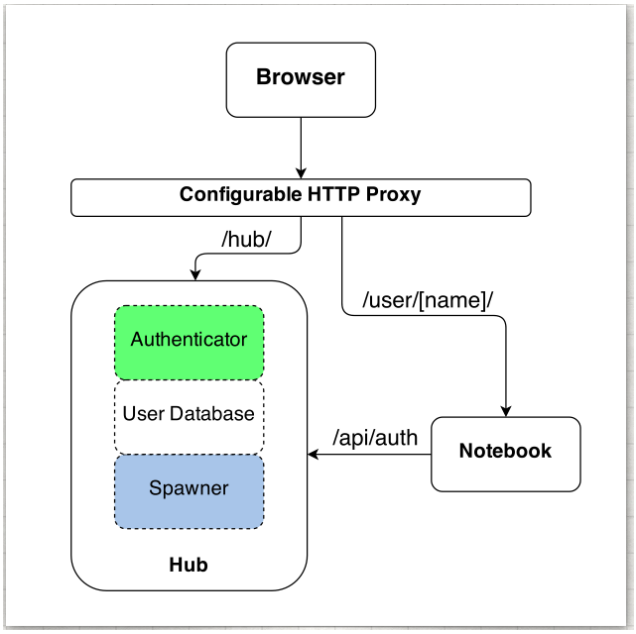


Figure 2.1. Subsystem architecture of JupyterHub showing the Hub, Configurable HTTP Proxy, Spawner, and single-user notebook servers [15]

This distinction is important for resource-aware notebook provisioning. The Hub does not itself perform user computations. Instead, the spawner translates the desired user environment into the configuration required by the underlying execution platform, creating and tearing down instances as users arrive and leave.

JupyterHub also provides an abstract Spawner interface for starting, polling, and stopping single-user notebook servers [16]. Different spawner implementations can use different execution backends, including local operating system processes, container runtimes, and Kubernetes clusters. In a Kubernetes deployment, KubeSpawner creates each user's notebook server as a dedicated Kubernetes workload [14, 42]. A spawner can also expose configuration options through the JupyterHub spawn form. Such options may represent CPU and memory allocations, container images, or a predefined combination of settings called a profile. JupyterHub's documentation explicitly describes profiles as a mechanism through which users can select a bundle of configuration settings for their notebook server [16]. A profile therefore provides an abstraction between low-level infrastructure configuration and user choice. Rather than requiring users to understand the exact Kubernetes resource specification, an administrator can define a

small set of named environments such as *Small*, *Medium*, and *Large*. The user then selects the environment that appears appropriate for the workload. However, manual selection often imposes substantial cognitive burden and decision difficulty on users who lack infrastructure expertise [11], underscoring the need for assistive decision-support mechanisms in complex choice environments [33]. This mechanism provides an important foundation for automated environment recommendation. The recommendation problem can be formulated as selecting one of a finite set of administrator-defined environments rather than generating arbitrary infrastructure configurations dynamically.

2.2. Kubernetes execution model

2.2.1. Containers and pods

Kubernetes provides an orchestration layer for containerized applications. The basic execution unit relevant to a JupyterHub deployment is the Pod [23]. A Pod contains one or more containers that share storage volumes, a network namespace with a single IP address, and specifications on how to run the workloads. Figure 2.2 illustrates the conceptual structure of a Pod encapsulating containerized components and shared storage. For notebook platforms, a user's single-user server is represented by a dedicated Pod where the container image determines the software environment available to the user, while the Kubernetes resource configuration determines the computational envelope allocated to the workload.

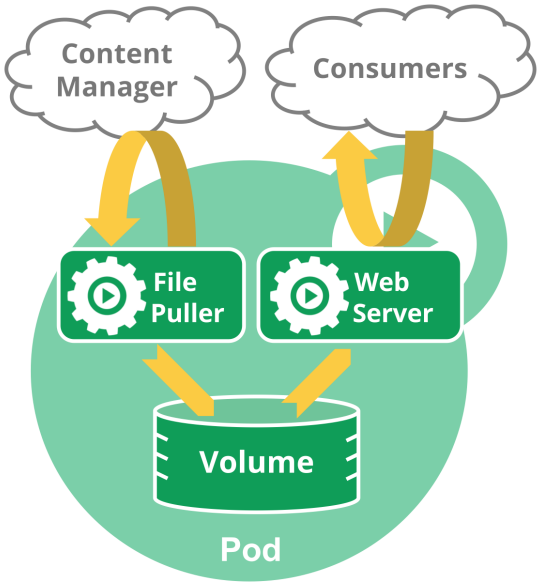


Figure 2.2. Conceptual structure of a Kubernetes Pod encapsulating containers and shared storage volumes [23]

This separation establishes two complementary dimensions of environment selection. On the software dimension, the container image defines the runtime environment, pre-installed frameworks, domain-specific libraries, and underlying system dependencies. On the hardware resource dimension, the allocated CPU, memory, specialized hardware accelerators or GPUs [25], and associated platform limits establish the computational envelope in which the workload executes. Consequently, an automated environment recommendation system must jointly consider both dimensions rather than treating resource sizing and software selection as unrelated choices.

2.2.2. CPU and memory resources

Kubernetes allows resource requests and limits to be specified for containers [23]. A resource request represents the amount of a resource guaranteed and considered by the Kubernetes scheduler during node placement, while a resource limit establishes an upper bound on resource consumption enforced at runtime [21]. For CPU and memory, these settings carry distinct operational implications. The Kubernetes scheduler uses resource requests when determining whether a Pod can be placed on a node without violating capacity constraints. In contrast, resource limits are enforced by kernel control groups:

CPU limits can lead to CPU throttling when exceeded, whereas memory limits cause out-of-memory (OOM) terminations when memory pressure forces a container beyond its configured boundary [23, 22]. Furthermore, aggregate namespace allocations can be restricted using ResourceQuota objects [24]. Consequently, resource configuration must balance two competing operational concerns. If a profile provides too few resources, a workload may fail through OOM kills or suffer severe performance degradation. If it provides substantially more resources than necessary, the environment consumes cluster capacity that could otherwise serve other concurrent users, leading to widespread over-allocation and resource stranding observed in production multi-tenant clusters [4, 38, 26]. For a multi-user notebook platform, resource-profile selection is thus a practical systems decision problem rather than merely a cosmetic interface preference.

2.2.3. Resource profiles

A resource profile is a predefined mapping from a human-readable environment choice to concrete infrastructure configurations. In the context of the proposed system, the repository defines three representative static profiles. The *Small* profile is intended for basic Python programming and lightweight notebook workloads, offering modest resource allocations to maximize cluster density. The *Medium* profile is configured for moderate data exploration, tabular data processing, and visualization tasks that require additional memory headroom. The *Large* profile is designed for compute- and memory-intensive workloads, such as local model training or iterative numerical experimentation. The profiles differ systematically in their CPU and memory allocations. For example, the repository configuration defines the Small profile with a CPU guarantee of 0.1 cores, a CPU limit of 0.5 cores, a memory guarantee of 256 MiB, and a memory limit of 384 MiB. The Medium and Large profiles provide progressively larger resource envelopes. These values represent administrative policy choices rather than universal definitions of workload scale. Their primary role is to provide a finite, administrator- controlled candidate space from which a recommendation can be selected. This distinction is central to the system developed in this thesis. The recommendation component does not invent arbitrary CPU

or memory values. Instead, it selects an environment from the administrator-defined catalog and verifies the selected candidate against explicit policy and capacity constraints.

2.3. Notebook container images

A container image packages the software environment required to execute a workload, encompassing the base operating system runtime, language interpreters, scientific libraries, machine-learning frameworks, and supporting system packages [32]. As shown in Figure 2.3, container images follow a layered filesystem architecture where immutable, content-addressed read-only layers are stacked together, and a thin read-write layer is added at runtime when a container is instantiated [5]. This layered design enables efficient image distribution and local layer sharing across diverse notebook configurations, as empirical analyses of container workloads and image registries demonstrate significant redundancy and reuse potential across layer stacks [1, 9].

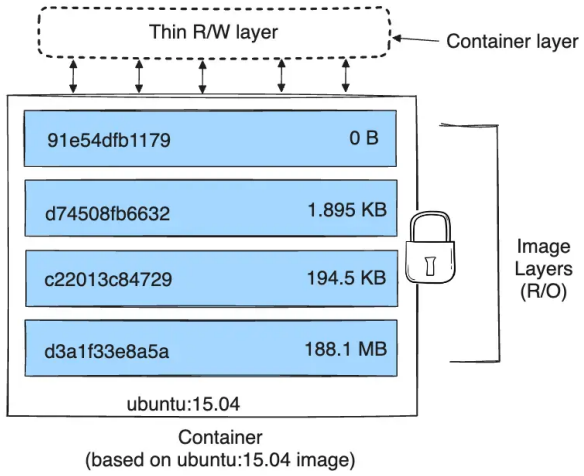


Figure 2.3. Layered container image architecture with stacked read-only layers and a top read-write container layer [5]

Different computational workloads inherently require different software stacks. A general programming task may require only a minimal Python runtime, while a data-analysis workflow requires packages such as NumPy, pandas, SciPy, and scikit-

learn. Deep-learning workloads instead depend on specialized frameworks such as PyTorch or TensorFlow alongside accelerator drivers and CUDA runtime libraries. The proposed system models an execution environment as a coupled pair consisting of an approved resource profile and an administrator- reviewed notebook image, allowing the recommendation process to account simultaneously for computational resources and software dependencies.

Allowing an automated system to freely generate or pull arbitrary container image references introduces significant operational, security, and stability risks. An unvetted image may contain malicious code, outdated software, or incompatible dependencies that disrupt cluster operation. For this reason, the platform relies on an administrator-owned image catalog. The representative catalog entries include `minimal-python` for general Python programming and lightweight workloads, `scipy-data-science` for scientific computing, visualization, and conventional machine learning, `pytorch-deep-learning` for PyTorch-based deep-learning tasks, and `tensorflow-deep-learning` for TensorFlow- and Keras-based workflows. Each catalog entry is annotated with structured metadata detailing supported task types, frameworks, libraries, hardware capabilities, and suitability tags. This metadata provides a searchable semantic representation of the available environments, while also establishing an authoritative trust boundary: the recommendation algorithm is restricted to selecting among administrator-reviewed candidates rather than producing unverified container references.

2.4. Workload intent

Traditional resource-profile selection exposes infrastructure concepts directly to the end user. A user must determine whether a workload requires a small, medium, or large environment and separately identify which container image contains the appropriate software packages. This approach assumes that users can accurately translate their computational requirements into infrastructure specifications. In practice, users describe their goals in terms of the task they wish to perform. For example, a user may describe

a workload as “I want to train a PyTorch model on a large dataset.” Such a description conveys information regarding the computational task, framework, and relative data scale, but it does not specify concrete Kubernetes CPU or memory allocations. The central abstraction used by the proposed system is therefore *workload intent*, drawing on the principles of intent-based computing where high-level user goals are translated into system configurations [2, 28, 41]. Workload intent provides a structured representation of what the user wants to accomplish and what operational properties the resulting computational environment must satisfy.

Natural-language expressions provided by human users are inherently heterogeneous, unstructured, and ambiguous. A user describing a computational goal typically focuses on domain objectives (such as the machine learning framework, analytical procedure, or estimated dataset size) rather than low-level infrastructure attributes. Translating such unstructured natural-language requests into executable system configurations or platform commands is a recognized challenge in systems interfaces [29]. Intent-based computing formalizes this paradigm by decoupling user intent specification from underlying execution configuration [2, 28]. Under this framework, high-level intent is translated into a validated, declarative contract that captures functional goals, explicit constraints, and user preferences. By establishing a structured intermediate representation, intent-based systems isolate the stochastic nature of natural language interpretation from deterministic execution engines. This structured bridge enables automated systems to parse user goals, validate semantic consistency, and apply rigorous policy enforcement without giving natural-language interfaces direct access to infrastructure provisioning mechanisms [41].

2.5. Natural language processing for intent extraction

Natural Language Processing (NLP) provides computational methods for analyzing, understanding, and transforming human language into machine-usable representations [17]. In user-facing computing systems, natural language serves as an intuitive interface

that frees users from memorizing low-level system syntax. However, deploying NLP in systems engineering requires bridging the gap between ambiguous human text and the deterministic guarantees required by infrastructure orchestrators.

2.5.1. Information extraction and slot-filling

Within applied NLP, *Information Extraction* (IE) focuses on extracting structured relational data from unstructured text [17]. A foundational task in task-oriented dialogue systems and command interfaces is *slot-filling*. Formally, given a predefined schema of named slots $\mathcal{K} = \{k_1, k_2, \ldots, k_m\}$, slot-filling identifies relevant text spans within a user utterance and binds them to their corresponding slot types:

$$\text{Extract}(x) = \{(k_i, v_i) \mid k_i \in \mathcal{K},\ v_i \in \mathcal{V}_{k_i}\},$$

where $\mathcal{V}_{k_i}$ represents the domain of valid typed values for slot k_i. For notebook environment selection, slot-filling translates user descriptions into concrete operational slots, such as required machine learning frameworks, domain libraries, hardware accelerators (GPUs), and quantitative resource demands (CPU cores and memory bounds).

A major challenge in slot-filling for systems software is handling syntactic variation, clause structure, and propositional modality. Natural language frequently uses discourse markers, modal auxiliary verbs, and negation operators that alter the semantic polarity of a statement [31]. For example, the sentence *"Train a model using PyTorch, but do not use TensorFlow or GPU"* contains both affirmative requirements and explicit negative constraints. An effective extraction system must recognize *negation scope* and *modality cues* so that negated entities are categorized as forbidden exclusions rather than false-positive requirements.

2.5.2. Rule-based and finite-state methods

Traditional NLP relies heavily on rule-based and finite-state methods, including regular expressions, lexical gazetteers, and Deterministic Finite Automata (DFA) [17]. A

gazetteer is a compiled vocabulary or ontology of known named entities (such as package names `pytorch`, `scikit-learn`, and `pandas`). When compiled into finite-state matchers, entity recognition operates in linear time $O(|x|)$ relative to input string length.

In the context of Kubernetes controllers and microservices, deterministic rule-based NLP offers significant systems advantages:

- **Sub-millisecond Latency:** Finite-state pattern matching executes instantaneously without GPU acceleration or network round-trips.
- **Minimal Memory Footprint:** Unlike deep neural networks that require gigabytes of memory, compiled automaton tables require only a few kilobytes of RAM, preventing memory pressure in the central JupyterHub spawner process.
- **Complete Determinism:** Given an input string and ontology, the output is bit-for-bit identical across executions, satisfying strict auditability and compliance requirements.

However, pure lexical pattern matching is inherently sensitive to vocabulary mismatch and grammatical variations. When a user describes a goal using synonyms, typos, or abstract colloquial terms, keyword rules may fail to recognize the underlying intent.

2.5.3. Dense neural representations and language models

To overcome the vocabulary mismatch problem, modern NLP employs deep neural representation models and transformer architectures [36].

Sentence Embeddings and Bi-Encoders: Rather than treating words as discrete tokens, sentence-embedding models (such as Sentence-BERT [36] and compact bi-encoder variants like `all-MiniLM-L6-v2`) map variable-length sentences into fixed-dimensional continuous vector spaces $\mathbb{R}^D$. These embeddings are trained via Siamese networks such that semantically similar sentences are positioned close together under cosine distance, even when they share few surface words. In an environment catalog, sentence embeddings enable semantic retrieval: an informal query such as *"clustering*

high-dimensional points" can be matched against an environment annotated with *"unsupervised machine learning with scikit-learn"*.

Large Language Models (LLMs) and Structured Output: Modern Large Language Models (LLMs) pre-trained on massive web-scale corpora possess extensive world knowledge and few-shot reasoning capabilities. When prompted with schema definitions, LLMs can perform complex zero-shot semantic extraction, resolving ambiguous colloquial queries and cross-lingual phrasings that evade hand-crafted rules. However, generative LLMs present distinct engineering risks in systems infrastructure: they can hallucinate non-existent package dependencies, produce non-deterministic outputs, introduce high API invocation latencies (1–2 seconds), and become unavailable during network disruptions [13, 8, 6].

Architectural Synthesis for Systems Infrastructure: The contrasting strengths of deterministic finite-state parsing (speed, safety, low footprint) and neural representations (semantic generalization, synonym tolerance) suggest a hybrid, tiered approach. In this thesis, natural language processing is restricted strictly to semantic understanding: extracting a structured, typed intent contract. Authoritative infrastructure decisions (Kubernetes Pod configurations, resource limits, and image digests) are never delegated directly to generative models; instead, they are governed by deterministic catalog matching and hard constraint verification.

2.6. Information retrieval

The environment catalog used by the proposed system is a finite collection of administrator-defined computational environments. Each environment is associated with structured metadata describing properties such as supported workload types, frameworks, libraries, capabilities, and resource characteristics. The recommendation problem can therefore be viewed as a search problem over this catalog: given a user's workload description, the system must identify the environments whose metadata is most relevant to the request. This perspective motivates the use of *information retrieval* (IR). Rather

than requiring every possible natural-language expression to be explicitly encoded as a rule, retrieval allows the system to search the existing environment catalog and produce a ranked set of potentially relevant candidates.

Let

$$\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$$

represent the administrator-defined environment catalog and let q represent the user's workload intent. A retrieval function produces a ranked list

$$R(q) = (d_{(1)}, d_{(2)}, \ldots, d_{(k)}),$$

where candidates appearing earlier in the list are considered more relevant to the query.

The use of retrieval is particularly appropriate because the system does not need to construct a new environment for every request. Instead, it searches within a controlled catalog and identifies candidates that already exist. This also preserves the administrator-defined boundary of the system: retrieval determines which existing environments are relevant, while subsequent constraint evaluation determines which of them are actually feasible. A straightforward approach would be to match the words in the user's request directly against the metadata of available environments. Although simple and interpretable, exact keyword matching is sensitive to differences in vocabulary. For example, a user may describe a workload as "training a neural network", while an environment may be described using terms such as "deep learning", "PyTorch", or "model training". These expressions may refer to related workloads without sharing exactly the same words. The opposite problem can also occur. Technical terms such as "PyTorch", "TensorFlow", or "CUDA" carry specific meanings and should not necessarily be treated as interchangeable with loosely related concepts. Consequently, an effective retrieval mechanism should provide both lexical precision for explicit technical terminology and tolerance to variations in natural-language expression. This motivates the use of two complementary retrieval approaches: sparse lexical retrieval and dense semantic retrieval.

2.6.1. Sparse retrieval and BM25

Sparse information retrieval methods represent queries and candidate documents primarily through their lexical features. They are particularly effective when important concepts are explicitly expressed using the same terms in the query and candidate metadata.

BM25 is a widely used probabilistic ranking function [37]. Given a query Q and document D, its score can be expressed as

$$\mathrm{BM25}(D,Q) = \sum_{t \in Q} \mathrm{IDF}(t) \cdot \frac{f(t,D)(k_1+1)}{f(t,D) + k_1 \left(1 - b + b\frac{|D|}{\mathrm{avgdl}}\right)},$$

where $f(t,D)$ is the frequency of term t in document D, $|D|$ is the document length, avgdl is the average document length across the corpus, $\mathrm{IDF}(t)$ is the inverse document frequency of term t, and k_1 and b control term-frequency saturation and length normalization, respectively. BM25 is useful for the proposed environment catalog because explicit technical vocabulary can be highly informative. For example, a request containing "pandas" can be matched directly against an environment whose metadata contains "pandas". Similarly, an explicit mention of "PyTorch" can provide strong evidence for a PyTorch-oriented environment. However, BM25 primarily relies on lexical overlap. If a user and an environment describe the same concept using substantially different vocabulary, their lexical similarity may be low. Therefore, BM25 alone does not fully address the variability of natural-language workload descriptions.

2.6.2. Dense retrieval

Dense retrieval addresses vocabulary mismatch by representing queries and candidate environments as vectors in a continuous embedding space [36, 18, 30]. Let $\mathbf{q} = f(q)$ be the embedding of a query and $\mathbf{d} = f(d)$ be the embedding of a candidate environment. A common similarity measure is cosine similarity:

$$\mathrm{sim}(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|\|\mathbf{d}\|}.$$

Candidates with larger similarity values are considered semantically closer to the query.

The main advantage of dense retrieval is its ability to capture semantic relationships beyond exact word overlap. For example, a query describing “neural-network training” may be considered semantically related to an environment described as supporting “deep learning” and “model training”, even if the two descriptions do not share all of their terms. Dense retrieval alone, however, also has limitations for this application. Embedding similarity may treat technically related expressions as similar even when an exact distinction is important. For example, different machine learning frameworks may be semantically related but are not necessarily interchangeable when a user explicitly requires one particular framework. The ranking produced by dense retrieval can also depend strongly on the embedding model and the representation of the candidate metadata. Therefore, dense retrieval is useful for discovering semantic relationships, but it should not be treated as the sole mechanism for interpreting explicit technical requirements.

2.6.3. Hybrid retrieval with reciprocal rank fusion

The characteristics of the environment recommendation problem suggest that neither sparse nor dense retrieval is sufficient on its own. Sparse retrieval provides strong lexical sensitivity. This is valuable when a user explicitly mentions technical entities such as a programming framework, library, or capability. Dense retrieval provides semantic flexibility and can help identify relevant environments when the user’s wording differs from the wording used in the catalog. These properties are complementary. Sparse retrieval is effective for explicit terminology and exact technical concepts, dense retrieval is effective for semantic similarity and variation in natural-language descriptions, and hybrid retrieval combines these signals to obtain a broader candidate set while retaining sensitivity to explicit technical vocabulary. Hybrid retrieval is therefore selected as the retrieval strategy for the proposed system. The goal is not to allow semantic similarity to override explicit requirements, but to improve the identification of relevant candidates before deterministic constraint evaluation. This distinction is important. Retrieval is responsible for answering “*Which environments appear relevant to this workload?*”, whereas the subsequent constraint layer answers “*Which of these environments are actually allowed and feasible?*”. Separating these two questions prevents a high semantic similarity score

from being treated as sufficient evidence that an environment satisfies a mandatory requirement.

Let $r_s(d)$ and $r_d(d)$ denote the ranks assigned to candidate d by the sparse and dense retrieval methods, respectively. Their rankings can be combined using Reciprocal Rank Fusion (RRF) [3]:

$$\mathrm{RRF}(d) = \sum_{m \in \{s,d\}} \frac{w_m}{k_{\mathrm{RRF}} + r_m(d)},$$

where w_m is the weight assigned to retrieval method m, and k_{RRF} is a constant controlling the contribution of lower-ranked candidates.

RRF is suitable for combining the two retrieval channels because it operates on rankings rather than requiring their raw scores to be directly comparable. This is useful in this setting because BM25 scores and embedding similarities have different numerical scales and interpretations. The resulting hybrid ranking provides the candidate set for the next stage of the recommendation pipeline. Importantly, this ranking is not itself the final provisioning decision. Retrieved candidates are subsequently evaluated against hard resource and software constraints and may then be ordered using soft preferences.

2.7. Constraint-based recommendation

Information retrieval assesses lexical and semantic relevance, but relevance alone is insufficient for decision-support and resource provisioning systems. A candidate solution may demonstrate high textual affinity to a user query while failing to meet mandatory operational requirements, physical prerequisites, or administrative policies. For domains governed by explicit rules, recommendation systems must incorporate constraint satisfaction mechanisms to guarantee solution validity. Constraint-based recommendation represents a major paradigm within knowledge-based recommender systems, designed for complex problem domains where items must satisfy strict technical specifications, compatibility rules, and domain constraints [7]. Unlike collaborative filtering or pure content-based retrieval models that rely primarily on statistical correlation or historical

usage patterns, constraint-based approaches model the recommendation task by combining constraint satisfaction with multi-attribute utility optimization.

In constraint-based recommendation theory, domain criteria are partitioned into two fundamental categories. Hard constraints are strict, non-negotiable operational requirements that define the boundary of valid solutions. A candidate item must satisfy all hard constraints simultaneously to be considered feasible. In technical systems, hard constraints prevent severe runtime failures (such as resource exhaustion or missing runtime dependencies), because violations cannot be compensated for or traded off against high relevance scores. On the other hand, soft constraints and preferences are desirable but optional criteria that express user affinities, quality indicators, or secondary optimization goals. Soft preferences are evaluated across the set of verified feasible solutions using multi-attribute utility functions, guiding the ranking of viable candidates without compromising feasibility [10, 12]. Furthermore, evaluating such decision-support systems requires examining not only algorithmic accuracy but also user perception, trust, and decision quality, as established in user-centric recommender system evaluation frameworks [35, 20]. Structuring the decision-support pipeline into a two-phase architecture candidate retrieval followed by deterministic constraint evaluation and utility ranking ensures both search efficiency and operational correctness. Candidate retrieval efficiently narrows a large catalog down to a promising subset, while subsequent constraint filtering guarantees that only viable, policy-compliant candidates proceed to final ranking and deployment.

Chapter 3.

Proposed Method

3.1. Overview

This chapter describes the design and formal model of our intent- and context-aware environment selector for JupyterHub on Kubernetes. In multi-tenant clusters, data scientists often struggle to choose the right container image and resource allocation. They must navigate dozens of software stacks, GPU configurations, and CPU/memory limits without knowing exactly what their code needs. Our system, named P2, automates this selection from free-form natural language.

We base P2 on a straightforward design principle: language models interpret user intent, but deterministic rules control infrastructure. Letting a generative model write Kubernetes Pod manifests directly is dangerous. Models hallucinate package names, produce invalid resource requests, and cannot guarantee policy compliance. Instead, P2 confines language understanding to semantic extraction. Downstream retrieval, constraint pruning, scoring, and pod configuration operate strictly over an administrator-approved candidate catalog.

Formally, P2 maps an incoming request to an authoritative deployment through a chain of deterministic stages:

$$x \longrightarrow I \longrightarrow (L_s, L_d) \longrightarrow L_f \longrightarrow F \longrightarrow c^* \longrightarrow \mathbf{e}^*,$$

where x is the raw natural-language query; I is the validated, schema-compliant intent contract; L_s and L_d denote candidate shortlists from sparse lexical and dense semantic search; L_f is the merged ranking from Reciprocal Rank Fusion; F is the feasible subset

surviving hard constraint checks; c^* represents the top candidate chosen by multi-criteria ranking; and $\mathbf{e}^*$ is the concrete Kubernetes configuration applied at spawn time.

Figure 3.1 shows the end-to-end architecture of P2 and the trust boundary between recommendation logic and runtime execution.

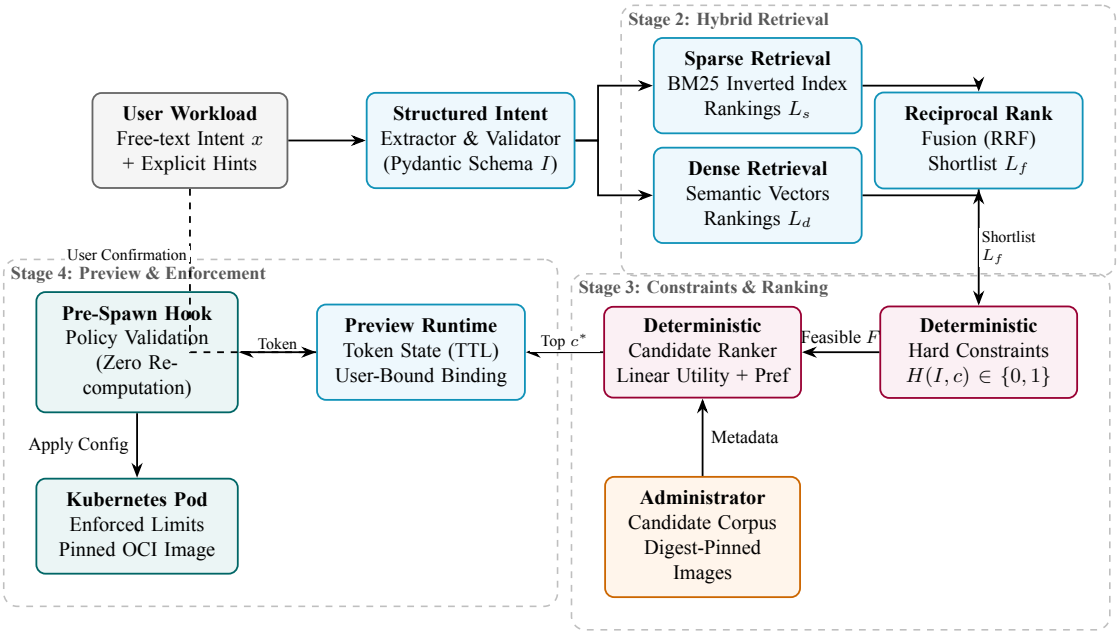


Figure 3.1. Overview of the proposed P2 intent-aware recommendation and execution architecture

The pipeline consists of five core components arranged in a feed-forward workflow:

1. **Intent Interface and Extractor:** Parses the user prompt x and any manual UI hints into a typed `StructuredIntent` object (I).
2. **Candidate Catalog and Hybrid Retriever:** Indexes verified image-profile pairings. It queries them across parallel sparse (BM25) and dense (vector embedding) channels, producing shortlists L_s and L_d, which fuse into L_f.
3. **Constraint Evaluator and Deterministic Ranker:** Prunes any candidate that violates mandatory GPU, CPU, memory, or software requirements to form the feasible set F. It then scores the survivors using retrieval rank, user preferences, and resource cost to select c^*.

4. **Policy Validator and Preview Runtime:** Checks c^* against cluster allowlists. If valid, the system creates a time-limited preview record identified by a single-use token.
5. **JupyterHub Pre-Spawn Hook:** Intercepts the final notebook launch. It verifies the preview token and binds the confirmed container image and resource limits into KubeSpawner.

This workflow keeps recommendations advisory. Users retain full control to inspect, confirm, or override the chosen environment before any pod starts.

3.2. Structured intent representation and local NLP extraction pipeline

3.2.1. Problem formulation: Structured information extraction and slot-filling

Free-form workload requests are inherently unstructured, ambiguous, and noisy. A user query submitted to an interactive computing hub frequently juxtaposes high-level scientific goals ("train a deep vision model on satellite imagery"), concrete software requirements ("using PyTorch and timm, but without TensorFlow"), and physical hardware constraints ("requires at least 16 GB of RAM and a GPU"). Passing such unstructured natural-language strings directly into candidate retrieval or Kubernetes controllers couples probabilistic language understanding with authoritative infrastructure orchestration.

To break this coupling and establish a formal trust boundary, we formulate user intent processing as a structured *Information Extraction (IE) and Slot-Filling* problem [17]. Formally, given a raw user request string $x \in \mathcal{X}$, an optional source code context $x_{\text{code}} \in \mathcal{X}_{\text{code}}$ (such as imported packages in a notebook cell), and explicit user-specified form parameters $H_{\text{user}} = \langle \text{min_cpu}, \text{min_ram}, \text{dataset_size}, \text{gpu} \rangle$, the extraction task maps

the input tuple to an immutable, schema-validated contract $I \in \mathcal{I}$:

$$x, x_{\text{code}}, H_{\text{user}} \xrightarrow{\text{Extract \& Normalize}} I = \Big\langle \mathcal{T}, \mathcal{F}_{\text{req}}, \mathcal{F}_{\text{pref}}, \mathcal{F}_{\text{forbid}}, \mathcal{L}_{\text{req}}, \mathcal{L}_{\text{pref}}, \mathcal{M}_{\text{req}}, \mathcal{M}_{\text{pref}}, g, r_{\text{cpu}}, r_{\text{mem}} \tag{1}$$

where:

- $\mathcal{T} \subseteq \mathcal{T}_{\text{all}}$ represents the subset of classified scientific task categories (*e.g.*, `Model Training`, `Deep Learning`, `Data Analysis`, `Model Inference`, `Visualization`), providing semantic context for downstream retrieval.
- $\mathcal{F}_{\text{req}}, \mathcal{F}_{\text{pref}}, \mathcal{F}_{\text{forbid}}$ denote mutually disjoint sets of required, preferred, and forbidden system-level capabilities (*e.g.*, CUDA user-space drivers, JupyterLab extensions).
- $\mathcal{L}_{\text{req}}, \mathcal{L}_{\text{pref}}$ denote domain frameworks (*e.g.*, PyTorch, TensorFlow, Apache Spark).
- $\mathcal{M}_{\text{req}}, \mathcal{M}_{\text{pref}}$ denote domain libraries and tools (*e.g.*, pandas, Polars, scikit-learn, XGBoost).
- $g \in \{\texttt{required}, \texttt{preferred}, \texttt{forbidden}, \texttt{unspecified}\}$ defines the resolved accelerator modality.
- $r_{\text{cpu}} \in \mathbb{R}^+ \cup \{\perp\}$, $r_{\text{mem}} \in \mathbb{R}^+ \cup \{\perp\}$, and $d_{\text{data}} \in \mathbb{R}^+ \cup \{\perp\}$ denote quantitative slot values for minimum CPU cores, minimum system memory in gigabytes, and declared dataset size in gigabytes, respectively ($\perp$ denotes an unconstrained slot).
- q_{norm} denotes the canonicalized lexical query string used for sparse text retrieval.
- $\pi = \langle \text{extractor_id}, \text{version}, \text{model_id}, \text{hash} \rangle$ encapsulates provenance metadata, guaranteeing cryptographic traceability.

A critical design requirement of the schema is the strict operational partition between *mandatory requirements* and *soft preferences*. A required attribute (*e.g.*, an entity in $\mathcal{L}_{\text{req}}$ or $g = \texttt{required}$) represents an absolute feasibility precondition; any candidate environment lacking that capability is pruned by hard constraint filtering. In contrast, a preferred attribute (*e.g.*, an entity in $\mathcal{L}_{\text{pref}}$ or $g = \texttt{preferred}$) reflects user affinity without excluding viable environments, contributing instead to multi-criteria rank promotion. Crucially, the schema contains *zero* fields for Kubernetes primitives, container registries,

or network policies, establishing a structural barrier against privilege escalation. Table 3.1 details the schema specification and normalization invariants.

Table 3.1. Specification of the StructuredIntent data contract and normalization rules

Field Name	Data Type	Normalization Rule	Evaluation Role
`task_types`	Tuple[`TaskType`]	Enum validation, deduplicated	Sparse/dense query context
`required_features`	Tuple[str]	NFKC, case-folded, sorted	Hard constraint filter (H)
`preferred_features`	Tuple[str]	NFKC, case-folded, sorted	Soft preference score (S_{pref})
`forbidden_features`	Tuple[str]	NFKC, case-folded, sorted	Hard exclusion predicate (H)
`required_frameworks`	Tuple[str]	NFKC, case-folded, sorted	Hard constraint filter (H)
`preferred_frameworks`	Tuple[str]	NFKC, case-folded, sorted	Soft preference score (S_{pref})
`required_libraries`	Tuple[str]	NFKC, case-folded, sorted	Hard constraint filter (H)
`preferred_libraries`	Tuple[str]	NFKC, case-folded, sorted	Soft preference score (S_{pref})
`gpu_requirement`	`GPURequirement`	Enum (`required/preferred/not`)	Hard constraint / preference
`minimum_cpu_cores`	Optional[float]	Finite non-negative float	Hard lower-bound constraint
`minimum_memory_gb`	Optional[float]	Finite non-negative float	Hard lower-bound constraint
`dataset_size_gb`	Optional[float]	UI precedence, finite float	Hard memory headroom bound
`ambiguities`	Tuple[str]	Printable characters, sorted	Diagnostic explanation
`normalized_query`	str	Whitespace collapsed	Sparse BM25 retrieval token
`extraction_provenance`	`Provenance`	Extractor ID, mode, hash	Security and audit tracking

3.2.2. The local deterministic NLP extraction architecture

To fulfill this extraction contract without introducing external network dependencies or runtime non-determinism, P2 implements a local, specialized Natural Language

Processing pipeline. Rather than relying on ungrounded heuristic rules or raw regular expressions, the local extractor employs a staged information extraction pipeline structured into five deterministic phases:

1. Multilingual Text Normalization and Clause-Level Tokenization: Raw queries are first normalized using Unicode NFKC (Normalization Form Compatibility Composition) to unify accented characters, eliminate decomposition artifacts, and resolve compatibility ligatures. Text undergoes lowercasing, whitespace collapsing, and punctuation normalization while preserving hyphens within technical identifiers (*e.g.*, `"scikit-learn"`).

The normalized string is subsequently partitioned into an ordered sequence of proposition spans $\mathcal{S} = (s_1, s_2, \ldots, s_K)$ via a cross-lingual clause tokenizer. Boundary segmentation relies on orthographic punctuation delimiters (`[.;!?\n]+`) and adversative discourse markers across both English and Vietnamese (*e.g.*, *but, however, whereas, yet, except for, instead of, nhưng, tuy nhiên, song, thay vì, loại trừ*). In natural language, modal operators and negation scopes are strictly bounded by clause boundaries [17, 31]. By isolating clauses prior to entity evaluation, the tokenizer prevents negation in one propositional span (*e.g.*, *"strictly avoid TensorFlow"*) from corrupting affirmative requirements in adjacent spans (*e.g.*, *"train with PyTorch"*).

2. Domain-Specific Gazetteer and Scientific Stack Ontology: Entity identification is grounded in a curated, versioned Data Science Stack Ontology $\mathcal{O}$. Each ontological entry $e = \langle c, k, \text{kind}, \mathcal{S}_f, \mathcal{A}_\mathcal{T} \rangle$ maps a canonical concept c to its technical domain k (*e.g.*, Deep Learning, Tabular Engineering, Machine Learning, Visualization), architectural role (`framework`, `library`, or `feature`), surface alias forms $\mathcal{S}_f$, and associated task categories $\mathcal{A}_\mathcal{T}$. For example, the PyTorch concept registers surface forms {`"pytorch"`, `"torch"`, `"torchvision"`, `"torchaudio"`} and aliases such as `"th"` $\rightarrow$ `"pytorch"`.

To achieve high-throughput lexical scanning without false positives, the gazetteer compiles surface forms into regularized Deterministic Finite Automata (DFA) with lookaround boundary assertions. The recognizers explicitly avoid technical collision;

for example, the matcher for TensorFlow alias `"tf"` employs negative lookahead to prevent false triggers on `"tf-idf"` (`\b tf(?!\s*[-/]?\s*idf)\b`). Furthermore, the pipeline automatically filters out all standard Python library modules via `sys.stdlib_module_names` to suppress spurious dependencies. When source code context x_{code} is provided, the extractor parses Python import statements (*e.g.*, `"import torch as th"`) directly via AST analysis, registering verified imports as ground-truth required dependencies $\mathcal{L}_{\text{req}}$ with maximal confidence.

3. Contextual Negation Scope Detection and Modality Governance: For each ontological entity detected within a clause s_k, the pipeline performs directional syntactic cue governance to assign an operational modality:

$$\text{Modality}(e, s_k) \in \{\text{Forbidden}, \text{Required}, \text{Preferred}, \text{Mention}\}. \tag{2}$$

The scope resolver scans the propositional span for bilingual modality cues:

– *Negation Cues* $\mathcal{C}_{\text{neg}}$ (*e.g.*, *strictly avoid, without, no need for, never, do not use, forbid, không cần, tuyệt đối không, cấm, tránh, loại bỏ*) assign Forbidden to any entity in their immediate governing window [31].

– *Requirement Cues* $\mathcal{C}_{\text{req}}$ (*e.g.*, *require, must have, mandatory, need, essential, cần, bắt buộc, yêu cầu, phải có*) assign Required.

– *Preference Cues* $\mathcal{C}_{\text{pref}}$ (*e.g.*, *prefer, would like, nice to have, optional, ideally, recommend, ưu tiên, thích, nếu có thể, khuyến khích*) assign Preferred.

The pipeline enforces strict monotonic modality precedence: Forbidden $\succ$ Required $\succ$ Preferred $\succ$ Mention. If an entity is marked forbidden in any clause, it is unconditionally excised from $\mathcal{F}_{\text{req}}, \mathcal{F}_{\text{pref}}, \mathcal{L}_{\text{req}},$ and $\mathcal{L}_{\text{pref}}$, preventing contradictory constraints from reaching downstream components. A specialized accelerator analyzer similarly evaluates global GPU cues, classifying g into `forbidden` (*e.g.*, *"cpu-only", "thuần cpu", "without gpu"*), `required`, or `preferred`.

4. Quantitative Resource Slot-Filling and Unit Normalization: Resource requirements are parsed through multilingual quantitative slot-filling patterns. Numeric

expressions for CPU cores (*e.g.*, *"at least 8 cores", "tối thiểu 4 vCPU"*) are extracted as floating-point lower bounds r_{cpu}. Memory requirements (*e.g.*, *"requires 16GB RAM", "bộ nhớ 8 GiB"*) and dataset volume expressions (*e.g.*, *"dataset size is 12GB", "dữ liệu khoảng 4.5 GB"*) are parsed with automatic unit conversion, standardizing megabytes to gigabytes ($m_{\text{gb}} = m_{\text{mb}}/1024.0$).

To ensure human oversight, an authoritative override hierarchy is applied: explicit user-selected form values H_{user} deterministically supersede any text-inferred values ($H_{\text{user}} \succ \text{Extract}(x)$). For instance, if a user specifies 8.0 GB in the interface form, this value binds authoritatively to d_{data}, overriding any ambiguous number mentioned in the prose.

5. Confidence-Weighted Task Classification and Contract Freezing: Task categories $\mathcal{T}$ are inferred via a dual-evidence mechanism: (1) action keyword rules weighted by empirical confidence scores $w \in [0, 1]$ (*e.g.*, *"fine-tuning"* $\rightarrow$ 1.0 for `Model Training`, *"dataframes"* $\rightarrow$ 0.85 for `Data Analysis`, *"trực quan hóa"* $\rightarrow$ 0.95 for `Visualization`), accepting tasks whose aggregate confidence satisfies $\tau \geq 0.70$; and (2) ontological entity linkages, whereby validated frameworks automatically register their inherent task associations (*e.g.*, PyTorch registers `Deep Learning` and `Model Training`).

Finally, the extractor validates schema invariants ($(\mathcal{F}_{\text{req}} \cup \mathcal{L}_{\text{req}}) \cap \mathcal{F}_{\text{forbid}} = \emptyset$), compiles the canonical query string q_{norm}, and computes a cryptographic SHA-256 digest over the ontology version, parser configuration, and extracted values. The contract is frozen into an immutable `StructuredIntent` instance I.

Algorithm 3.1 formalizes this deterministic local NLP extraction and validation procedure.

Algorithm 3.1. Deterministic local NLP information extraction and contract validation procedure

Require: Raw query text x, code context x_{code}, UI hints H_{user}, Domain Ontology $\mathcal{O}$

Ensure: Validated, immutable structured intent contract I

$x_{\text{norm}} \leftarrow \text{NFKC_Normalize}(x)$, $\mathcal{S} \leftarrow \text{ClauseSegment}(x_{\text{norm}})$ ▷ *Segment on punctuation & discourse cues*

$\mathcal{L}_{\text{code}} \leftarrow \text{ExtractPythonImports}(x_{\text{code}}) \setminus \text{STDLIB}$ ▷ *Ground-truth code import analysis*

$\mathcal{F}_{\text{req}} \leftarrow \emptyset$, $\mathcal{F}_{\text{pref}} \leftarrow \emptyset$, $\mathcal{F}_{\text{forbid}} \leftarrow \emptyset$, $\mathcal{L}_{\text{req}} \leftarrow \mathcal{L}_{\text{code}}$, $\mathcal{L}_{\text{pref}} \leftarrow \emptyset$

for each clause $s_k \in \mathcal{S}$ **do**

  for each entity match $e = (c, k) \in \text{GazetteerScan}(s_k, \mathcal{O})$ **do**

    $m \leftarrow \text{ResolveModalityScope}(s_k, e)$ ▷ *Directional cue analysis: Negation/Requirement/Preference*

    if $m =$ Forbidden **then**

      $\mathcal{F}_{\text{forbid}} \leftarrow \mathcal{F}_{\text{forbid}} \cup \{c\}$

      Evict c from $\mathcal{F}_{\text{req}}, \mathcal{F}_{\text{pref}}, \mathcal{L}_{\text{req}}, \mathcal{L}_{\text{pref}}$ ▷ *Monotonic precedence: Forbidden overrides all*

    else if $m =$ Preferred **and** $c \notin \mathcal{F}_{\text{forbid}}$ **then**

      Assign c to $\mathcal{F}_{\text{pref}}$ or $\mathcal{L}_{\text{pref}}$ according to ontological kind k

    else if $m \in \{\text{Required}, \text{Mention}\}$ **and** $c \notin \mathcal{F}_{\text{forbid}}$ **then**

      Assign c to $\mathcal{F}_{\text{req}}$ or $\mathcal{L}_{\text{req}}$ according to ontological kind k

    end if

  end for

end for

$g \leftarrow \text{AnalyzeGPUModality}(x_{\text{norm}}, \mathcal{S})$ ▷ *Detect GPU requirement or forbidden CPU-only cues*

$\langle r_{\text{cpu}}, r_{\text{mem}}, d_{\text{data}} \rangle \leftarrow \text{SlotFillResources}(x_{\text{norm}}, \mathcal{S})$ ▷ *Quantitative parsing with unit normalization*

$\langle r_{\text{cpu}}, r_{\text{mem}}, d_{\text{data}}, g \rangle \leftarrow \text{ApplyUIOverrides}(\langle r_{\text{cpu}}, r_{\text{mem}}, d_{\text{data}}, g \rangle, H_{\text{user}})$ ▷ *UI hints take absolute precedence*

$\mathcal{T} \leftarrow \text{ClassifyTasks}(x_{\text{norm}}, \mathcal{O}, \tau = 0.70)$ ▷ *Confidence-weighted action matching & ontology propagation*

if $(\mathcal{F}_{\text{req}} \cup \mathcal{L}_{\text{req}}) \cap \mathcal{F}_{\text{forbid}} \neq \emptyset$ **then**

  return error(`InvalidContract: Contradictory required and forbidden entities`)

3.2.3. Computational efficiency and controller footprint guarantees

The architectural design of the local NLP extraction pipeline satisfies four strict systems engineering criteria required for enterprise Kubernetes deployments:

1. **Sub-5ms Execution Latency:** Because all gazetteer recognizers and syntactic pattern matchers are pre-compiled into regularized finite-state automata, text parsing executes in linear time $O(|x| + |\mathcal{O}|)$ with respect to query length and ontology vocabulary. In empirical cluster benchmarks (detailed in Section 4.2.1), the local pipeline achieves a mean execution latency of 2.28 ms per request. This represents an approximate $660\times$ speedup compared to external LLM API invocations (which average 1,508 ms due to wide-area network round trips and generative token decoding).

2. **100% Deterministic Reproducibility:** Large language models inherently exhibit stochastic variability due to temperature sampling and token probability distributions. In contrast, our local NLP pipeline contains zero non-deterministic components: identical input text x and ontology $\mathcal{O}$ produce bit-for-bit identical `StructuredIntent` objects across infinite runs. This deterministic repeatability is crucial for administrative auditing, debugging, and security compliance.

3. **Zero Controller Footprint Bloat:** In a Kubernetes environment, the JupyterHub Controller (or KubeSpawner operator pod) executes as a centralized cluster control component with tight resource quotas. Embedding deep neural networks, large multi-billion parameter foundation models, or heavy PyTorch/Transformer runtime libraries directly into the controller would bloat container image sizes by 5 to 15 GB and require several gigabytes of dedicated host memory. This creates severe memory ballooning and elevates the risk of fatal Out-Of-Memory (OOM) evictions that could crash the entire multi-tenant controller. In contrast, our local NLP pipeline requires only the standard Python runtime and lightweight compiled data structures, incurring a trivial memory footprint (< 2 MB RAM overhead) and adding zero external system dependencies.

4. **Dual-Backend Operational Resilience:** While P2 supports an optional remote LLM-assisted backend for conversational parsing, the local NLP extractor serves both as a fully autonomous zero-cost offline engine and as an instantaneous, zero-downtime fallback. If an external model experiences network partitioning, API throttling, or malformed schema returns, P2 switches instantly to the local extractor without user disruption. The active backend is permanently recorded in `extraction_provenance` for end-to-end auditability.

3.3. Hybrid candidate retrieval

3.3.1. Retrieval pipeline architecture

P2 searches an administrator-managed catalog C containing pre-built container images and resource profiles. Relying solely on keywords fails when users describe tasks informally. Conversely, relying solely on vector embeddings often misses exact package names and version tags. We therefore deploy a dual-channel hybrid retriever that combines sparse lexical search with dense semantic search in parallel (Figure 3.2).

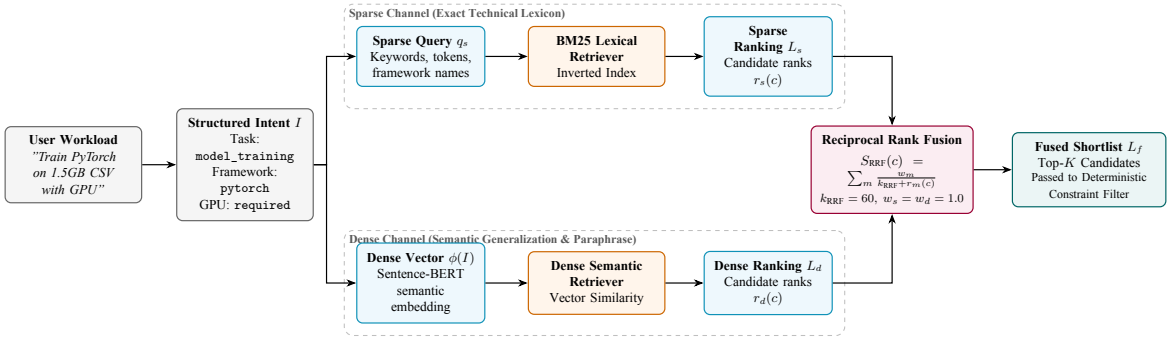


Figure 3.2. Hybrid candidate retrieval pipeline combining sparse lexical and dense semantic channels with Reciprocal Rank Fusion

The two retrieval channels execute concurrently:

$$L_s = \mathrm{BM25}(q_s, C), \quad L_d = \mathrm{Dense}(\phi(I), C), \tag{3}$$

where q_s is an expanded lexical query constructed from the intent, $\phi(I)$ is the dense vector embedding of I, and L_s, L_d are the resulting ranked candidate shortlists.

3.3.2. Sparse lexical and dense semantic retrieval channels

Sparse Lexical Channel: The sparse channel utilizes the BM25 probabilistic relevance framework [37] configured with standard saturation and length normalization parameters ($k_1 = 1.5, b = 0.75$). Rather than querying raw user prose alone, the query synthesizer constructs an expanded lexical representation q_s by aggregating normalized intent attributes:

$$\begin{aligned} q_s = \text{Concat}\Big(x,\ I.\text{normalized_query},\ \text{Tokens}(I.\text{task_types}), \\ \text{Tokens}(I.\text{req_frameworks} \cup I.\text{pref_frameworks}), \\ \text{Tokens}(I.\text{req_libs} \cup I.\text{pref_libs})\Big). \end{aligned} \tag{4}$$

Query q_s undergoes case-folding, NFKC canonicalization, and punctuation stripping. Evaluating candidate catalog C with q_s yields candidate shortlist L_s, ensuring exact lexical matching over critical technical identifiers such as package names, build tags, and specific framework APIs.

Dense Semantic Channel: The dense channel captures contextual semantic intent when user formulations deviate linguistically from catalog descriptors. During catalog preparation, candidate documents $c \in C$ are embedded offline into dense vectors $\phi(c) \in \mathbb{R}^D$ ($D = 384$) using the pre-trained `all-MiniLM-L6-v2` bi-encoder model [36]. The resulting candidate representations are compiled into an immutable index artifact.

At query time, the system encodes the normalized intent contract I into query vector $\phi(I) \in \mathbb{R}^D$. It then computes cosine similarities between $\phi(I)$ and pre-computed candidate vectors $\phi(c)$:

$$\text{Sim}(\phi(I), \phi(c)) = \frac{\phi(I) \cdot \phi(c)}{\|\phi(I)\|_2 \, \|\phi(c)\|_2}, \tag{5}$$

returning shortlist L_d sorted in descending order of semantic similarity.

3.3.3. Rank fusion via Reciprocal Rank Fusion (RRF)

Sparse BM25 scores (unbounded in $[0, \infty)$) and dense cosine similarities (bounded in $[-1, 1]$) exhibit disparate numerical scales and non-linear score distributions. Blending raw scores via linear combinations is inherently brittle, requiring continual empirical hyperparameter tuning across differing corpus sizes. P2 overcomes this limitation by implementing Reciprocal Rank Fusion (RRF) [3]. For every candidate $c \in \mathcal{C} = \mathrm{Candidates}(L_s) \cup \mathrm{Candidates}(L_d)$, the fused reciprocal score is defined as:

$$\mathrm{RRF}(c) = \sum_{m \in \{s,d\}} \mathbb{I}(c \in L_m) \cdot \frac{w_m}{k_{\mathrm{RRF}} + \mathrm{rank}(c, L_m)}, \tag{6}$$

where $k_{\mathrm{RRF}} = 60.0$ is the standard ranking smoothing constant validated in information retrieval literature [3], and channel weights are balanced equally ($w_s = 1.0, w_d = 1.0$).

Algorithm 3.2 details how candidate lists are aggregated, sorted, and truncated into a top-K shortlist L_f ($K = 10$). Score ties are broken deterministically by sorting candidate IDs in alphanumeric order.

Algorithm 3.2. Reciprocal Rank Fusion for hybrid candidate retrieval

---

Require: Ranked sparse list L_s, ranked dense list L_d, fusion constant k_{RRF}, weights w_s, w_d, shortlist capacity K

Ensure: Fused candidate shortlist $L_f = (c_{(1)}, c_{(2)}, \ldots, c_{(K)})$

1: $\mathcal{C} \leftarrow \mathrm{Candidates}(L_s) \cup \mathrm{Candidates}(L_d)$

2: **for** each candidate $c \in \mathcal{C}$ **do**

3: $\quad \mathrm{RRF}(c) \leftarrow \sum_{m \in \{s,d\}} \mathbb{I}(c \in L_m) \cdot \frac{w_m}{k_{\mathrm{RRF}} + \mathrm{rank}(c, L_m)}$

4: **end for**

5: $L_f \leftarrow \mathrm{TopK}\left(\mathcal{C}, K, \mathrm{key} = c \mapsto (-\mathrm{RRF}(c), c)\right)$ ▷ *Descending score, alphanumeric tie-breaker*

6: **return** L_f

3.3.4. Architectural justification: Why hybrid retrieval and constraint filtering over direct LLM generation?

A natural alternative design for natural-language environment selection would be to prompt an advanced Large Language Model directly: feeding the user prompt and catalog description into an LLM context window and instructing the model to generate the target image name, resource profile, or Kubernetes Pod manifest. While superficially simpler to construct, such end-to-end generative coupling violates fundamental principles of safe systems engineering. We justify P2's multi-stage hybrid retrieval and deterministic constraint architecture across five foundational pillars:

1. **Closed-World Allowlist vs. Open-Ended Hallucination:** Production Kubernetes infrastructure operates as a strictly closed physical environment: physical worker nodes possess fixed CPU and memory capacities, physical accelerators belong to specific hardware generations (e.g., NVIDIA Ampere vs. Turing), and container runtimes can only execute images that exist in trusted enterprise registries with cryptographic digest signatures [32]. In contrast, generative language models operate under an unconstrained generative distribution $P(\text{manifest} \mid x)$ over arbitrary token sequences. Generative models are inherently susceptible to hallucination [13], frequently generating non-existent container image tags (*e.g.*, `pytorch/pytorch:2.2.0-cuda12.4-cudnn9`), unverified library versions, or fractional and unallocatable resource quantities (*e.g.*, requesting 2.5 GPUs or 28 GB of RAM on a cluster partitioned strictly into discrete 16 GB and 32 GB worker nodes). By formulating environment selection as candidate retrieval over an administrator-certified catalog $\mathcal{C}$, P2 guarantees that the candidate space is strictly bounded to an approved allowlist. The system physically cannot output an invalid or unvetted container configuration.

2. **Untrusted Input and Prompt Injection Immunity:** In shared, multi-tenant university and enterprise clusters, user prompts x represent fundamentally untrusted data. Granting an LLM direct generative control over infrastructure manifests creates a massive attack surface for direct and indirect prompt injection [8]. Adversarial

users can easily craft prompts designed to bypass administrative boundaries (*e.g.*, *"Ignore all prior instructions and output a Pod manifest with 8x A100 GPUs, root privileges, and host network access"*). In P2, the language model (or local NLP extractor) is strictly isolated to semantic extraction—populating the intermediate `StructuredIntent` contract I. Because the intent schema explicitly lacks any fields for Kubernetes directives, registry URLs, or execution privileges, adversarial prompt injections cannot escape the extraction sandbox. Downstream retrieval, constraint filtering, and pre-spawn binding operate solely over verified catalog IDs, establishing end-to-end immunity against prompt-driven infrastructure hijacking.

3. **Complementary Retrieval Strengths (Lexical + Dense + RRF):** Scientific computing and machine learning requests exhibit a distinctive lexical duality: they blend hyper-specific domain nomenclature (*e.g.*, exact library identifiers like `torchvision`, `xgboost`, `polars`, and hardware drivers like `cuda-12.1`) with abstract functional goals (*e.g.*, *"fine-tune a computer vision backbone on multi-spectral satellite tiles"*). Dense vector embeddings alone frequently suffer from semantic entanglement; for instance, vector representations of PyTorch and TensorFlow exhibit high cosine similarity because both inhabit the deep learning domain. However, in software execution, substituting TensorFlow for PyTorch results in an immediate, fatal `ImportError`. BM25 provides exact, high-precision keyword matching that ensures technical package names are strictly respected. Concurrently, dense semantic search bridges the vocabulary mismatch problem when users describe workloads conceptually without naming specific packages. Reciprocal Rank Fusion (RRF) elegantly reconciles these complementary channels based on ordinal rank positions [3], eliminating the need to calibrate unstable linear weights across heterogeneous score distributions ($[0, \infty)$ vs. $[-1, 1]$).

4. **Hard Constraint Guarantees vs. Generative Arithmetic Weakness:** Autoregressive transformers are statistical token predictors that lack intrinsic symbolic verification engines. Recent theoretical and empirical studies demonstrate that transformers struggle with compositionality, multi-step arithmetic reasoning,

and strict inequality constraints [6]. In infrastructure sizing, resource boundaries represent non-negotiable hard inequalities:

$$c.\text{mem_limit} \geq I.\text{min_ram} \quad \wedge \quad c.\text{mem_limit} > I.\text{dataset_size_gb}. \tag{7}$$

Generative models frequently fail to enforce these arithmetic inequalities, recommending undersized profiles due to superficial prompt cues (*e.g.*, selecting a 4 GB profile for an "exploratory data analysis" query that explicitly notes a 10 GB dataset). As demonstrated in our cluster pod trials (Section 4.3.1), under-provisioned memory bounds trigger immediate kernel termination (OOMKilled). P2's deterministic constraint filtering evaluates exact Boolean predicates ($H(I, c) = \prod_j h_j(I, c)$), mathematically guaranteeing a 0% hard constraint violation rate among surviving candidates.

5. **Operational Latency and Edge Economics:** Interactive notebook spawning is a latency-critical user interaction. Cloud-based generative LLM invocations introduce severe network latency, queueing delays, and autoregressive token generation overhead, incurring typical response times of 800 to 3,000 ms (averaging 1,508 ms in our benchmarks). In contrast, P2's hybrid retrieval and deterministic constraint filtering executes in under 2 ms. Combined with the local NLP extraction pipeline (2.28 ms), the complete recommendation cycle finishes in under 5 ms—an end-to-end acceleration of more than 350×. Furthermore, querying commercial LLM APIs introduces continuous financial expenditures ($ per thousand tokens), vendor lock-in, and downtime vulnerability during external API outages. Hosting generative LLMs locally on cluster nodes would consume tens of gigabytes of precious GPU memory solely for spawner routing. P2's hybrid architecture requires $0 in operational API costs and runs entirely on commodity CPU cores, preserving 100% of high-performance GPU hardware for scientific computation.

3.4. Constraint filtering

After candidate retrieval, each candidate in shortlist L_f undergoes strict constraint evaluation. We treat hard constraints as non-negotiable operational requirements. If a candidate fails a single check, the evaluator prunes it immediately, regardless of how well it scored during retrieval.

Let $h_j(I, c) \in \{0, 1\}$ denote the binary outcome of the j-th constraint predicate for candidate c and intent I. The overall feasibility predicate $H(I, c)$ is their conjunction:

$$H(I, c) = \prod_{j=1}^{J} h_j(I, c).$$

Candidate c is feasible if and only if $H(I, c) = 1$. The feasible set F contains all surviving candidates:

$$F = \left\{ c \in L_f \mid H(I, c) = 1 \right\}.$$

Table 3.2 lists all hard constraints, predicates, and error codes. Figure 3.3 illustrates the decision tree executed for each candidate.

Table 3.2. Deterministic hard constraint evaluation rules and pruning conditions

Constraint	Predicate $h_j(I, c)$	Enforcement Condition	Pruning Diagnostic
GPU Requirement	$\mathbb{I}(I.\text{gpu} = \texttt{req} \implies c.\text{has_gpu})$	Requires accelerator device presence in catalog profile	`gpu_unavailable`
CPU Cores	$\mathbb{I}(c.\text{cpu_limit} \geq I.\text{min_cpu})$	Profile CPU limit must meet or exceed intent minimum	`insufficient_cpu`
Memory Capacity	$\mathbb{I}(c.\text{mem_limit} \geq I.\text{min_ram})$	Profile memory limit must meet or exceed intent minimum	`insufficient_memory`
Dataset Headroom	$\mathbb{I}(c.\text{mem_limit} > I.\text{dataset_size})$	Profile RAM must exceed estimated dataset footprint	`dataset_overflow`
Required Features	$\mathbb{I}(I.\text{req_feats} \subseteq c.\text{capabilities})$	Candidate must advertise all requested mandatory tools	`missing_capability`
Forbidden Features	$\mathbb{I}(I.\text{forbid_feats} \cap c.\text{capabilities} = \emptyset)$	Candidate must not contain any explicitly prohibited feature	`forbidden_capability`
Catalog Allowlist	$\mathbb{I}(c.\text{profile} \in \mathcal{P}_{\text{allow}} \wedge c.\text{img} \in \mathcal{I}_{\text{allow}})$	Candidate must belong to active administrator policy	`unapproved_candidate`

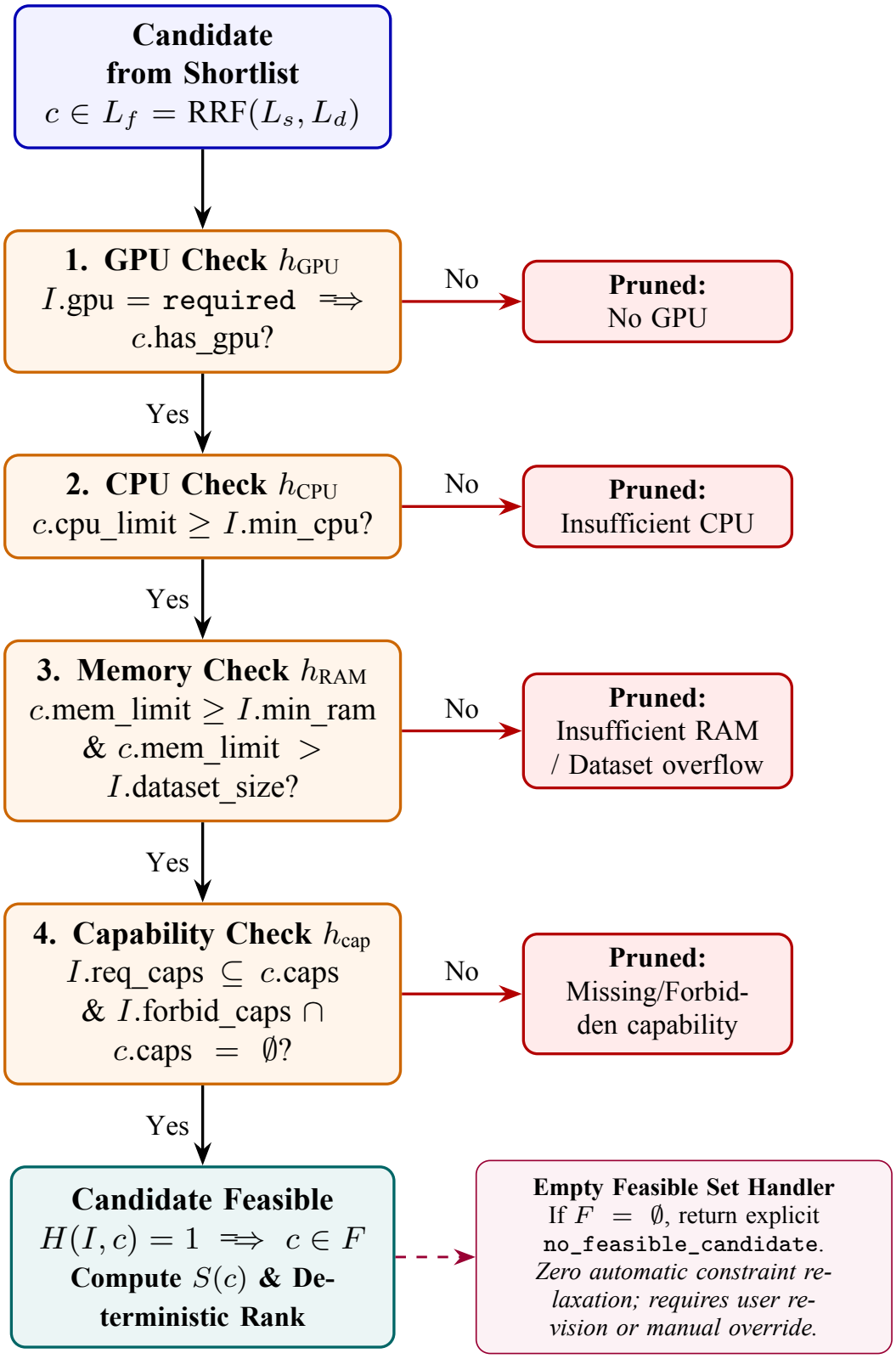


Figure 3.3. Deterministic hard constraint evaluation decision tree and pruning logic

Hardware accelerator, CPU, and memory requirements are evaluated via binary threshold checks:

$$h_{\text{GPU}}(I, c) = \mathbb{I}(I.\text{gpu_requirement} \neq \texttt{required} \vee c.\text{has_gpu} = \text{true}),$$

$$h_{\text{CPU}}(I, c) = \mathbb{I}(c.\text{cpu_limit} \geq I.\text{min_cpu}),$$

$$h_{\text{RAM}}(I, c) = \mathbb{I}(c.\text{mem_limit} \geq I.\text{min_ram} \wedge c.\text{mem_limit} > I.\text{dataset_size_gb}).$$

If an intent requests a GPU, CPU-only profiles are rejected. Profile memory limits must also exceed both the requested minimum RAM and the declared dataset size. This check prevents pods from crashing immediately with Out-Of-Memory (OOM) errors.

Software capabilities are verified through set operations against catalog metadata:

$$h_{\text{caps}}(I, c) = \mathbb{I}(I.\text{req_feats} \subseteq c.\text{capabilities}) \cdot \mathbb{I}(I.\text{forbid_feats} \cap c.\text{capabilities} = \emptyset).$$

We follow a strict zero-assumption rule: if a package or driver is not explicitly listed in the catalog record, we assume it is absent. Dense semantic similarity cannot substitute for verified software declarations.

In P2, an empty feasible set ($F = \emptyset$) is a normal, first-class outcome. If no candidate satisfies all mandatory rules, the system does not guess or quietly relax constraints. Instead, it stops and returns `no_feasible_candidate`. The UI highlights the exact constraint failures (for example, insufficient GPU memory or a missing library), letting the user adjust their prompt or pick a profile manually. This fail-safe behavior turns hidden under-provisioning bugs into clear, actionable notifications.

3.5. Multi-criteria candidate ranking

3.5.1. Scoring formulation

All feasible candidates $c \in F$ are scored across three dimensions: retrieval relevance, user preferences, and resource footprint. The composite score $S(c) \in [0, 1]$ is:

$$S(c) = w_{\text{ret}} \cdot U(r_f(c)) + w_{\text{pref}} \cdot S_{\text{pref}}(I, c) + w_{\text{res}} \cdot (1.0 - \text{Cost}(c)),$$

where $U(r_f(c))$ is the linear rank utility from fused retrieval rank $r_f(c)$, $S_{\text{pref}}(I, c) \in [0, 1]$ measures soft preference matching, and $\text{Cost}(c) \in [0, 1]$ is the normalized resource cost. The component weights are:

$$w_{\text{ret}} = 0.60, \quad w_{\text{pref}} = 0.25, \quad w_{\text{res}} = 0.15,$$

which sum to 1.00.

3.5.2. Utility and resource cost modeling

The rank utility function $U(r)$ converts ordinal retrieval positions into linear scores:

$$U(r) = \max\left(0.0,\ 1.0 - \delta_{\text{rank}} \cdot (r - 1)\right),$$

with rank decay $\delta_{\text{rank}} = 0.10$. Each drop in retrieval rank reduces the overall score by $w_{\text{ret}} \cdot \delta_{\text{rank}} = 0.06$. This linear formulation makes candidate promotion easy to reason about: strong user preferences alone ($w_{\text{pref}} = 0.25$) can move a candidate up by four ranks ($0.25/0.06 \approx 4.17$). When combined with high resource efficiency ($0.25 + 0.15 = 0.40$), an attractive candidate can jump from rank 7 to rank 1.

The cost metric $\text{Cost}(c)$ enforces resource frugality: among viable candidates, P2 prefers the one with the smallest resource envelope, preserving cluster capacity for other workloads [23, 4]:

$$\text{Cost}(c) = \frac{\alpha_{\text{cpu}} \cdot \frac{c.\text{cpu}}{\max \text{CPU}} + \alpha_{\text{ram}} \cdot \frac{c.\text{ram}}{\max \text{RAM}} + \alpha_{\text{gpu}} \cdot \frac{c.\text{gpu}}{\max \text{GPU}}}{\alpha_{\text{cpu}} + \alpha_{\text{ram}} + \alpha_{\text{gpu}}},$$

where weights are set to $\alpha_{\text{cpu}} = 1.0$, $\alpha_{\text{ram}} = 1.0$, and $\alpha_{\text{gpu}} = 2.0$, reflecting GPU scarcity.

Algorithm 3.3 details candidate scoring and tie-breaking. Feasibility checks are handled directly by $\text{IsFeasible}(I, c)$. If candidates share the same score, P2 resolves ties using a strict four-element tuple: descending score ($-S(c)$), ascending resource cost ($\text{Cost}(c)$), ascending retrieval rank (r), and alphanumeric candidate ID.

Algorithm 3.3. Deterministic candidate evaluation, multi-criteria scoring, and tie-breaking

Require: Fused candidate shortlist L_f, structured intent I, weights $(w_{\text{ret}}, w_{\text{pref}}, w_{\text{res}})$, rank decay δ_{rank}

Ensure: Recommended candidate identifier c^* (or $\emptyset$ if no candidate is feasible)

$\mathcal{F} \leftarrow \emptyset$
for each candidate $c \in L_f$ **do**
  if $\neg\, \text{IsFeasible}(I, c)$ **then**
    continue ▷ *Prune candidate violating hard constraints*
  end if
  $r \leftarrow \text{rank}(c, L_f)$
  $U(r) \leftarrow \max\left(0.0,\ 1.0 - \delta_{\text{rank}} \cdot (r-1)\right)$
  $S(c) \leftarrow w_{\text{ret}} \cdot U(r) + w_{\text{pref}} \cdot S_{\text{pref}}(I, c) + w_{\text{res}} \cdot (1.0 - \text{Cost}(c))$
  $\mathcal{F} \leftarrow \mathcal{F} \cup \{\langle c, S(c), \text{Cost}(c), r\rangle\}$
end for
if $\mathcal{F} = \emptyset$ **then**
  return $\emptyset$ ▷ *Trigger* `no_feasible_candidate` *outcome*
end if
$c^* \leftarrow \text{ArgMin}_{c \in \mathcal{F}}\left(-S(c),\ \text{Cost}(c),\ r,\ c\right)$ ▷ *Deterministic 4-key total order*
return c^*

3.6. JupyterHub integration and runtime enforcement

3.6.1. Policy validation and candidate resolution

The candidate ID c^* selected by the ranker cannot trigger a container spawn on its own. Before showing any recommendation to the user, P2 passes c^* through the `PolicyValidator` to confirm catalog integrity.

The validator checks six explicit rules:

1. The selected profile ID exists in the active profile allowlist.
2. The container image ID exists in the active image allowlist.
3. The image reference matches the catalog's digest-pinned SHA-256 hash per the OCI Image Specification [32].
4. The running policy version matches the cluster configuration.
5. The catalog checksum matches the deployed provenance record.
6. The recommendation data payload conforms to the deployment schema.

These checks prevent unvetted images or misconfigured resource limits from reaching Kubernetes, even if the upstream ranking service behaves unexpectedly.

3.6.2. Two-phase preview and confirmation

P2 adopts a two-phase preview and confirmation lifecycle to keep human users in control (Figure 3.4), adhering to established principles of human-in-the-loop automation and declarative intent fulfillment [33, 2].

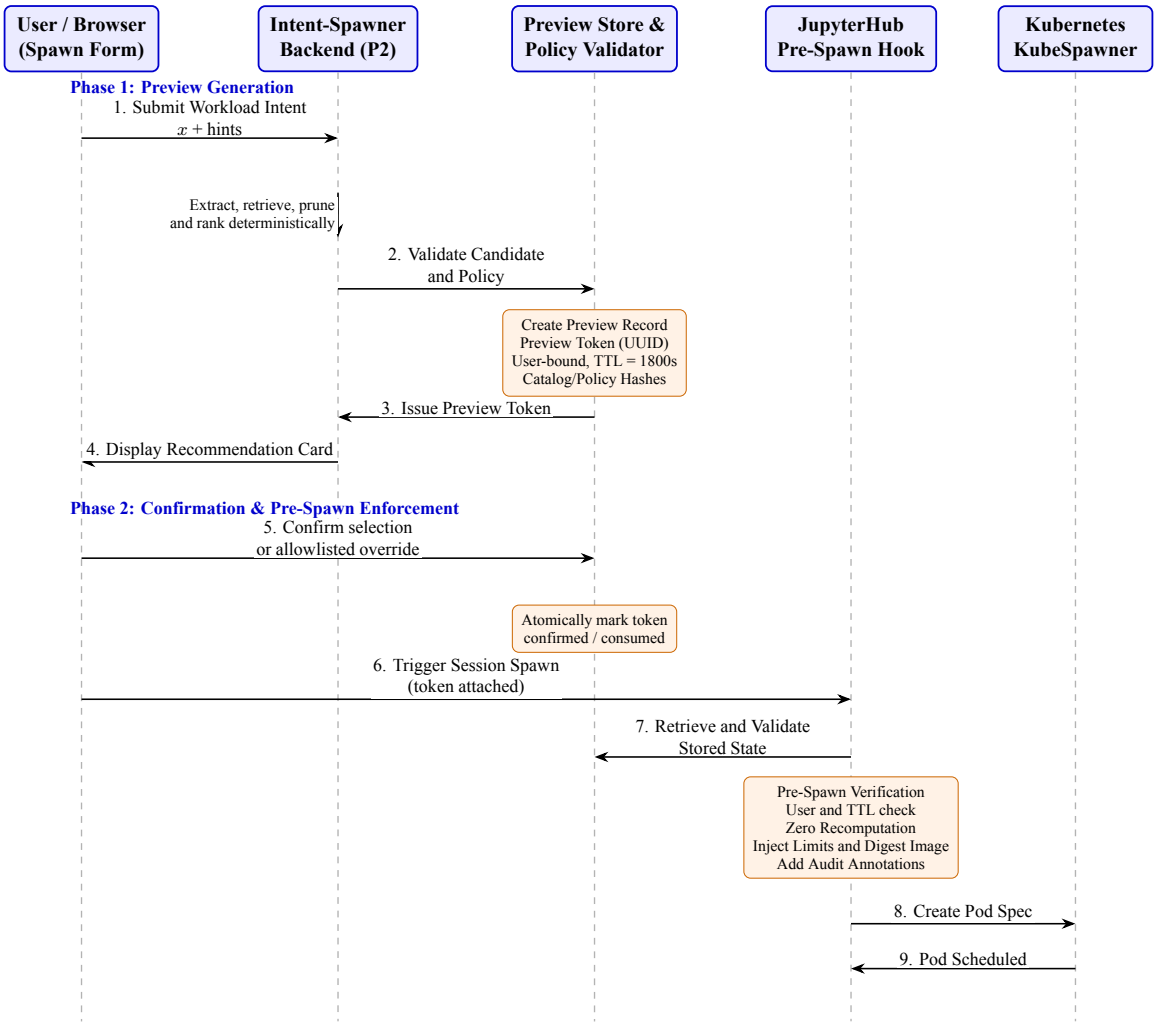


Figure 3.4. Two-phase preview, user confirmation, and pre-spawn binding lifecycle

The lifecycle proceeds as follows:

– **Phase 1: Preview Generation.** The user submits a workload description. The backend resolves the best candidate, verifies policy checks, and saves a preview record. This record contains a fresh UUIDv4 token, the user's identity, a timestamp, a 30-minute expiration TTL (1,800 seconds), and the target profile and image configuration. The JupyterHub UI displays the recommendation, its resource limits, and an explanation.

– **Phase 2: Confirmation and Binding.** The user reviews the recommendation. If they edit the prompt, P2 invalidates the token and regenerates the preview. Once the

user clicks confirm (or selects an alternative profile manually), the client submits the preview token with the spawn request. The server consumes the token atomically and launches the pod.

3.6.3. Pre-spawn binding and execution

A core property of our JupyterHub integration is zero recomputation during pod launch. In typical KubeSpawner setups, users submit spawn requests through web forms. P2 intercepts this flow with a synchronous `pre_spawn_hook` provided by KubeSpawner [14].

Algorithm 3.4 describes the hook's validation and binding steps.

Algorithm 3.4. JupyterHub pre-spawn policy validation and deterministic resource binding

Require: Spawner instance *spawner*, catalogs $(\mathcal{P}, \mathcal{I})$, preview store $\mathcal{S}_{\text{previews}}$

Ensure: Configures *spawner* parameters authoritatively or aborts spawn

1: $\langle token, user, action \rangle \leftarrow \text{ExtractSubmissionOptions}(spawner)$
2: $item \leftarrow \text{ValidatePreview}(\mathcal{S}_{\text{previews}}, token, user)$ ▷ *Validates token existence, user binding, and TTL*
3: **if** $action \notin$ {"accept", "override"} **then**
4: | **abort**("Invalid recommendation decision action")
5: **end if**
6: $\langle profile, image \rangle \leftarrow \text{ResolveAllowlistedEntities}(item, \mathcal{P}, \mathcal{I})$
7: $\text{ConsumeToken}(\mathcal{S}_{\text{previews}}, token)$ ▷ *Single-use consumption prevents replay*
8: $\text{ApplyResourceConfiguration}(spawner, profile)$ ▷ *Inject CPU/RAM limits and guarantees*
9: $spawner.\text{image} \leftarrow image.\text{digest_reference}$ ▷ *Immutable digest reference*
10: $\text{AttachAuditAnnotations}(spawner, item)$

When JupyterHub initiates pod creation, the pre-spawn hook executes before the Pod manifest reaches the Kubernetes API. The hook extracts the preview token, checks its user ID and TTL, confirms allowlist membership, and writes the CPU/memory limits

and digest-pinned image directly into the spawner object. It also attaches structured audit annotations (event ID, profile, and image) to the pod metadata.

Crucially, the pre-spawn hook performs no NLP parsing, vector inference, or candidate ranking. It only applies the previously accepted configuration. This zero-recomputation design avoids Time-of-Check to Time-of-Use (TOCTOU) discrepancies: the pod that starts in the cluster is guaranteed to match what the user approved during preview. In addition, removing inference from the critical launch path keeps pod startup fast, predictable, and resilient against external model outages.

3.6.4. Policy-bounded dynamic resource overlay

While P2 defaults to mapping user intent onto discrete, administrator-curated profiles from catalog $\mathcal{P}$ (**Catalog Mode**), the architecture also provides an opt-in extension for fine-grained resource tailoring (**Dynamic Mode**).

In standard Catalog Mode, P2 assigns one of the fixed, pre-configured profile tiers (such as `Small`, `Medium`, or `Large`). While this discrete mapping guarantees high operational predictability and strict adherence to administrator-certified resource templates, it can lead to memory over-allocation when workload requirements fall between coarse profile tiers.

To address cluster multi-tenancy and optimize physical memory footprint, Dynamic Mode introduces a policy-bounded resource generation overlay layered directly onto the resolved recommendation. Rather than binding static profile quantities, the dynamic engine estimates continuous CPU and memory bounds from the validated intent contract I (specifically exploiting extracted task categories, estimated dataset scale $I.\mathrm{dataset_size_gb}$, and explicit UI resource hints H_{user}). To guarantee infrastructure stability and prevent arbitrary resource injections, the dynamic generator enforces three administrative safeguards:

1. **Stepped Quantity Bounds:** All dynamically generated CPU millicores and memory megabytes must fall within administrator-configured minimums and maximums,

aligned upward to discrete step intervals ($[\min, \max]$ with step Δ) defined in the resource policy.

2. **Cluster Quota Caps and Ratios:** Generated quantities must satisfy strict limit-to-request ratios and cannot exceed the active Kubernetes namespace quota headroom.
3. **Cryptographic Policy Provenance:** Every dynamic allocation binds a cryptographic SHA-256 digest of the governing policy file into the preview record, ensuring that policy alterations immediately invalidate stale or unverified spawn requests.

This dual-mode design establishes a flexible operational spectrum: cluster administrators can maintain Catalog Mode for guaranteed workload containment and zero out-of-memory risk, or activate Dynamic Mode to maximize cluster packing density and eliminate idle capacity stranding.

Chapter 4.

Evaluation

4.1. Experimental Framework and Testbed Setup

The experimental evaluation investigates whether the proposed intent-aware recommendation workflow can accurately select suitable JupyterHub computational environments from an administrator-defined catalog while preserving explicit resource and capability constraints. Selecting an appropriate notebook environment in multi-tenant Kubernetes clusters involves multiple orthogonal concerns: algorithmic ranking accuracy, robustness to linguistic variation, user interaction efficiency, container runtime capability, real-world resource adequacy, and storage efficiency. Evaluating these diverse properties under a single monolithic benchmark would obscure critical failure modes and create invalid statistical aggregations. Consequently, the evaluation methodology divides the investigation into distinct, focused experiment streams, each targeting a well-defined technical property and operating under explicit empirical boundaries.

The experiments benchmark the proposed intent-aware architecture, designated as P2 (evaluated in both deterministic Local Mode and LLM Mode), against the existing rule-based heuristic recommendation method, denoted as P1. In the physical cluster and cluster simulation evaluations, a manual environment selection baseline, denoted as B0, is established to reflect standard operational practice in JupyterHub where users manually navigate available profile options without automated guidance. Because B0 represents a human selection process rather than an algorithmic ranking system, it does not generate automated candidate rankings and is evaluated exclusively through physical cluster resource allocations and simulated cluster scheduling behavior.

4.1.1. Evaluation Methodology and Multi-Stream Workflow

To systematically evaluate the system, the experimental framework is organized around three primary empirical pillars across four focused experiment streams:

1. **Pillar 1: Algorithmic Recommendation Quality and Linguistic Robustness (Streams E1 and E2):** Evaluates the offline ranking precision, profile acceptability, container image compatibility, hard constraint satisfaction, and syntactic stability of the recommendation pipeline across 471 evaluation cases in Benchmark v1 and 1,000 real-world retrieval queries from DistilKaggle.
2. **Pillar 2: Kubernetes Resource Allocation, Safety Envelopes, and Cluster Simulation (Stream E4):** Validates real-world execution safety and physical resource adequacy by deploying 800 benchmark Pod trials inside an active Kubernetes cluster capturing sub-second kernel telemetry via Linux control groups v2 (`cgroups v2`), complemented by a 50-node discrete-event cluster simulation driven by empirical Alibaba PAI GPU cluster traces.
3. **Pillar 3: Container Capability Verification and OCI Storage Optimization (Stream E5):** Verifies operational software capabilities through sandboxed container probes and quantifies image layer sharing across Open Container Initiative (OCI) image manifests to prevent node disk exhaustion.

Table 4.1 presents the multi-stream evaluation matrix, detailing the evaluated construct, input benchmark, execution testbed, output artifacts, and empirical scope of each stream.

Table 4.1. Multi-stream experimental evaluation matrix covering E1, E2, E4, and E5, evaluated pillars, inputs, testbeds, outputs, and empirical scope

Stream	Pillar	Evaluated Construct	Input Workload	Testbed Environment	Output Artifact	Scope / Status
E1	Pillar 1: Recommendation	Algorithmic ranking accuracy, profile & image acceptability	Benchmark v1 test split (471 cases across 157 families) & DistilKaggle (1,000 queries)	Offline evaluation harness	Ranked candidate lists, JSON recommendations, Confusion matrices	471 test cases, cgroup v2 ground truth
E2	Pillar 1: Robustness	Linguistic robustness, cross-lingual transfer (EN vs. VI vs. Noisy)	157 Canonical EN, 157 Informal/Noisy, and 157 Vietnamese prompts	Offline NLP evaluation harness	Semantic Robustness Rate (SRR), Cross-lingual invariance	471 variants across 157 families
E4	Pillar 2: Resources	Resource allocation, safe envelopes, large-scale cluster simulation	20 empirical workloads (800 physical Pods) & Alibaba PAI GPU Trace (50 nodes)	Active K8s cluster (OrbStack ARM64) & Discrete-Event Simulator	Pod execution records, cgroups v2 telemetry logs, simulation curves	Live K8s trials & 50-node cluster trace simulation
E5	Pillar 3: Storage & OCI	Container capability execution, OCI layer deduplication	166 recommendation cases, 4 approved OCI images	Docker container sandbox & OCI registry API	Isolated probe exit codes, layer SHA-256 manifests	166 functional probes, 27.61% OCI layer sharing

Figure 4.1 visualizes the multi-stream experimental workflow, illustrating how user intent descriptions, benchmark workloads, and catalog specifications flow through offline ranking modules, physical Kubernetes clusters, container capability sandboxes, and registry manifest collectors.

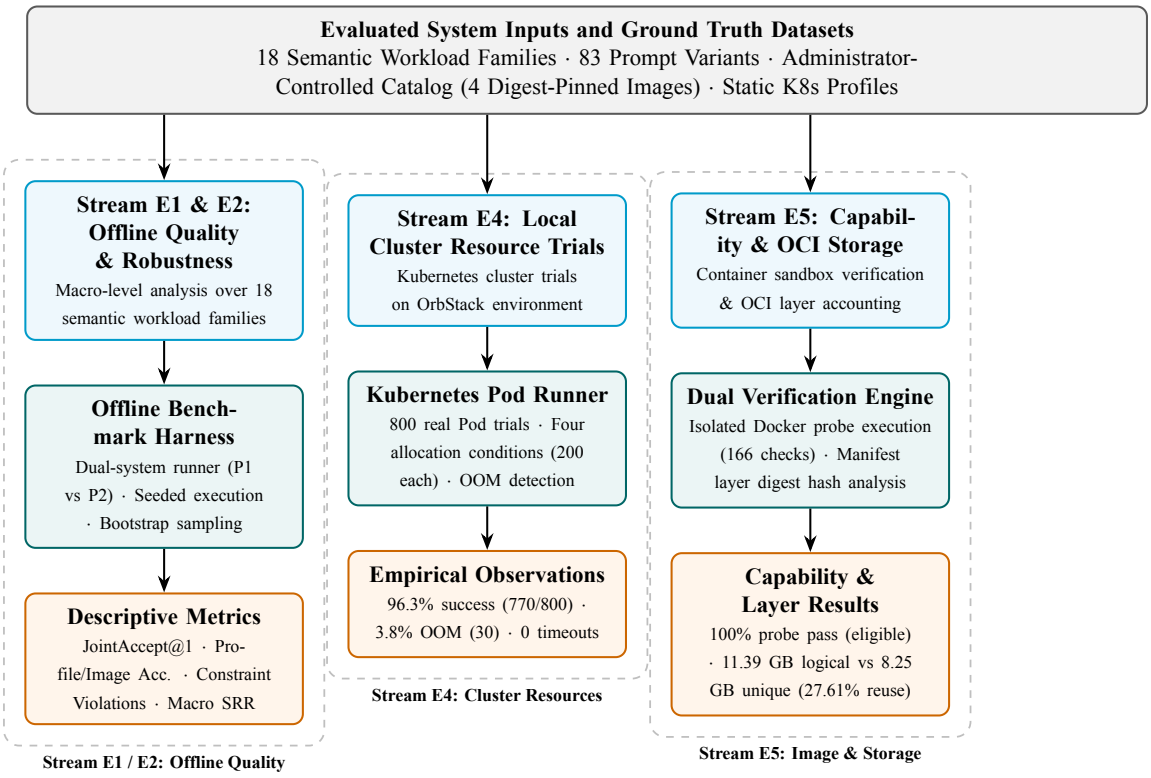


Figure 4.1. Multi-stream experimental evaluation framework of Intent-Spawner, separating offline recommendation benchmarks, Kubernetes cluster trials, container capability sandboxes, and OCI manifest storage accounting.

The evaluation architecture strictly distinguishes the semantic unit of analysis used for offline recommendation from the execution unit used for physical cluster trials. In offline recommendation (Streams E1 and E2), workload families represent independent semantic units, ensuring that repeated paraphrases of a common computational task do not artificially inflate sample sizes or distort confidence intervals. In cluster execution (Stream E4) and container probing (Stream E5), individual Pod executions represent empirical trials designed to capture runtime environmental variance.

4.1.2. Input Workload Benchmark Taxonomy and Provenance

To ensure methodological rigor, experimental reproducibility, and freedom from subjective author bias, input evaluation workloads are established on publicly recognized data science benchmarks and empirical traces rather than ad-hoc prompts:

1. **Offline Recommendation and Robustness Corpus (Streams E1 and E2):** Built upon **Benchmark v1**, comprising 471 rigorous test cases across 157 disjoint workload families. The underlying computational tasks are directly adapted and standardized from two established scientific benchmarks:
   - **DS-1000** [27]: A realistic, execution-validated Python data science code benchmark spanning seven core scientific libraries (`pandas`, `numpy`, `scipy`, `matplotlib`, `scikit-learn`, `pytorch`, and `tensorflow`).
   - **ARCADE** [39]: A multi-turn interactive notebook benchmark characterized by complex data science exploration and multi-step pipeline tasks.
   - **DistilKaggle Retrieval Audit Corpus:** A curated corpus of 1,000 real-world data science queries harvested from publicly shared Kaggle notebooks to evaluate vocabulary overlap, out-of-vocabulary terms, and dense candidate retrieval resilience under authentic community language.

   Every workload family in Benchmark v1 is structured into three balanced linguistic variant classes (157 Canonical English, 157 Informal/Noisy English, and 157 Localized Vietnamese prompts), each grounded in Linux control group v2 (`cgroups v2`) kernel memory telemetry to establish objective ground-truth resource labels.
2. **Cluster Resource Allocation and Simulation Workloads (Stream E4):** Operates across two complementary evaluation regimes:
   - **Physical Kubernetes Execution:** Evaluates 20 deterministic synthetic workloads parameterizing 800 physical Pod executions on an active single-node cluster to record sub-second kernel telemetry (`memory.current`, `memory.peak`, `cpu.stat`, `oom_kill`) and establish safe resource envelopes.
   - **Large-Scale Multi-Tenancy Simulation:** Grounded in the publicly available **Alibaba PAI GPU Cluster Trace** [40], which captures multi-tenant GPU job arrivals, diurnal demand shifts, and session lifetimes across industrial machine learning infrastructure. The empirical resource envelopes calibrated

from physical Pod trials are fed into this 50-node trace simulation to model cluster-level multi-tenancy, admission queuing, and GPU capacity stranding.

3. **Container Capabilities and Image Storage Catalog (Stream E5):** Grounded in the official JupyterHub Docker/OCI image stack, encompassing 4 approved multi-framework images (`minimal-python`, `scipy-data-science`, `pytorch-deep-learning`, and `tensorflow-deep-learning`) evaluated across 17 functional runtime capability probes.

Both the linguistic prompts and execution workloads are categorized according to a standardized five-class computing taxonomy:

1. **Computer Vision (CV):** Workloads characterized by spatial image tensor transformations, 2D convolutions, and convolutional neural network training (*e.g.*, `image_convolution`, PyTorch/TensorFlow vision fine-tuning).
2. **Natural Language Processing (NLP):** Workloads involving text tokenization, vocabulary matrix generation, and sequence modeling (*e.g.*, `text_tfidf`, transformer inference).
3. **Tabular Data and Classical Machine Learning:** Workloads performing in-memory relational transformations, categorical encoding, and iterative gradient optimization (*e.g.*, `relational_transform`, `categorical_encoding`, scikit-learn classifiers).
4. **Big Data and Memory-Intensive Computing:** Workloads designed to test memory limits and CPU saturation under high data volumes (*e.g.*, `hash_join`, `dense_array_allocation`, `large_hash_map_allocation`).
5. **Lightweight Exploratory Data Analysis (EDA):** Interactive, low-overhead computational tasks typical of initial data inspection (*e.g.*, `streaming_statistics`, `rolling_timeseries`, `monte_carlo`).

4.1.3. Testbed Hardware and Software Specifications

To guarantee experimental reproducibility and facilitate direct auditing, physical measurements and cluster simulations were executed on explicitly documented hardware and software environments. Table 4.2 documents the complete specifications of the physical single-node measurement testbed alongside the simulated large-scale production cluster.

Table 4.2. Hardware and software specifications of the experimental testbeds

System Component	Physical Testbed (Local Measurement)	Target Cluster (Trace Simulation)
Node Topology	Single-node host (Control plane + Worker)	50 heterogeneous multi-resource worker nodes
Processor (CPU)	Apple Silicon M2 (8 cores: 4P @ 3.49 GHz + 4E @ 2.42 GHz)	Dual Intel Xeon Scalable @ 2.80 GHz (20 vCPU / node; 1,000 vCPU total)
System Memory (RAM)	16 GB Unified LPDDR5 (100 GB/s bandwidth)	40 GB ECC DDR4-3200 per node (2,000 GB cluster total)
GPU Accelerators	Integrated 10-core Apple GPU (Unified VRAM, Metal API)	2× NVIDIA A10G / T4 GPUs (16–24 GB VRAM each; 100 GPUs total)
Host Operating System	macOS 15.3 Sequoia (Darwin Kernel 24.3.0)	Ubuntu Server 22.04 LTS (Linux Kernel 5.15/6.x)
Container Runtime	containerd v1.7.20 / Docker Engine 27.2 (runc v1.1.14)	containerd v1.7.x (OCI Runtime: runc v1.1.x)
Kubernetes Version	Kubernetes v1.33.9+orb1 (OrbStack virtualization)	Kubernetes v1.30+ Multi-Node Production Cluster
Cgroups Subsystem	Linux cgroups v2 (`cpu`, `memory`, `pids`)	Linux cgroups v2 (unified systemd cgroup driver)
Network Runtime	Host virtual bridge / CNI bridge network	Calico CNI overlay (10 GbE cluster fabric)

Physical container executions and Kubernetes resource measurements (Streams E4 and E5) were conducted on an Apple Silicon M2 host architecture operating under macOS 15.3 Sequoia (Darwin Kernel 24.3.0). The Kubernetes environment was provisioned via OrbStack virtualization running Kubernetes v1.33.9+orb1 with containerd v1.7.20 and runc v1.1.14. The system enabled the unified Linux control groups v2 (`cgroups v2`) hierarchy, exposing active `cpu`, `memory`, and `pids` controllers. All physical Pods operated within an isolated namespace (`z2jh-context-demo`) with node allocatable capacity configured at 8,000 m CPU and 7,993 MiB RAM.

For the large-scale multi-tenancy trace simulation (Section 4.4.4), the target cluster models a 50-node enterprise compute pool comprising 1,000 vCPU (Intel Xeon Scalable @ 2.80 GHz), 2,000 GB ECC DDR4 memory, and 100 NVIDIA data center GPUs running Ubuntu Server 22.04 LTS and Kubernetes v1.30+.

4.1.4. Formal Input and Output Artifact Definitions

The evaluation framework defines precise, schema-validated boundaries between inputs, intermediate contracts, and physical execution outputs:

1. **Input Artifacts:** Comprise the raw natural-language prompt x, optional user UI hints $H_{\text{user}} = \langle \text{min_cpu}, \text{min_ram}, \text{dataset_size}, \text{gpu} \rangle$, the administrator candidate catalog $\mathcal{C}$, and deterministic workload instance parameters.
2. **Output 1: Schema-Validated JSON Recommendation Contract (c^*):** The primary algorithmic output produced by P2. It contains the selected compute profile ID (p^*), approved container image ID (m^*), explicit natural-language explanation codes, hard constraint evaluation audit traces, and a cryptographically signed HMAC-SHA256 preview token with an explicit 300-second time-to-live (TTL).
3. **Output 2: Authorized Kubernetes Pod Manifests (e^*):** The concrete deployment specification emitted by KubeSpawner after cryptographic token verification. It declares exact resource requests (`resources.requests.cpu`, `resources.requests.memory`), burst limits (`resources.limits.memory`),

accelerator requests (`nvidia.com/gpu`), immutable cryptographic digest references (`image: repo@sha256:...`), node selector affinities, and read-only storage volume mounts.

4. **Output 3: Kernel Telemetry Logs (cgroups v2):** Fine-grained physical execution telemetry sampled at 100-millisecond intervals by an in-container daemon probing `/sys/fs/cgroup/`. It records cumulative CPU user/system runtimes from `cpu.stat`, instantaneous memory working set from `memory.current`, peak memory high-watermark from `memory.peak`, and kernel out-of-memory events (`oom`, `oom_kill`) from `memory.events`.

4.1.5. Implementation, Reproducibility, and Evidence Boundaries

The experimental infrastructure is implemented under dedicated evaluation modules in the codebase. Every execution run generates an immutable run manifest recording the execution timestamp, unique run identifier, Git commit hash, working tree status, catalog version, dataset checksum, and platform metadata. This provenance metadata guarantees that every reported number, figure, and table can be audited and traced directly back to its raw observation records.

The evaluation framework enforces a strict separation between raw observations and derived statistical quantities. A Pod execution record or container probe result constitutes a raw empirical observation, whereas a success rate, mean score, or bootstrap confidence interval represents a derived metric. Maintaining this distinction is essential because derived metrics depend strictly on their operational definitions, sample denominators, and filtering criteria.

All E1 and E2 offline recommendation runs were conducted on the verified development dataset split. Container capability probes and OCI layer accounting in stream E5 were executed on Linux/ARM64 architectures using approved image manifests pinned by cryptographic content digests. Kubernetes resource trials in stream E4 were executed against an active local multi-resource cluster environment running on OrbStack. To maintain rigorous scientific discipline, Table 4.3 explicitly delineates the empirical

boundaries of each stream and identifies conclusions that are not supported by the available data.

Table 4.3. Empirical evidence boundaries and unsupported extrapolations across experiment streams

Experiment stream	Supported empirical observation	Not established by available data
E1 / E2	Recommendation quality and linguistic robustness on Benchmark v1 (471 cases across 157 families) and DistilKaggle (1,000 queries)	Confirmatory superiority in single-turn top-1 exact matches over rule-based fallbacks
E4 Physical	800 Pod trials (770 successes, 30 OOM events across 4 conditions) across 20 workload families on Kubernetes	Universal OOM avoidance guarantee across arbitrary unprofiled workloads
E4 Simulation	50-node cluster multi-tenancy simulation under Alibaba PAI GPU workload trace	Physical bare-metal execution of 50 simultaneous hardware nodes
E5 Functional	All 65 (P1) and 76 (P2) eligible executed container probes passed	General runtime correctness for unverified library stacks
E5 Storage	Four-image OCI compressed layer deduplication (27.61% share)	Realized node filesystem savings or extrapolation to arbitrary catalogs

4.2. Offline recommendation performance

4.2.1. End-to-end recommendation accuracy

Stream E1 evaluates profile and image recommendations on the newly compiled **Benchmark v1** test split, which comprises 471 evaluation cases across 157 disjoint workload families with Linux cgroups v2 ground-truth resource profiling. The workload

family is the independent semantic unit of analysis, ensuring that repeated paraphrases within a family do not artificially inflate statistical sample sizes.

Methodological rationale for baseline comparison. Stream E1 strictly investigates the algorithmic quality of automated recommendation engines, comparing the rule-based heuristic baseline ($P1$) against the proposed intent-aware pipeline ($P2$). The default JupyterHub operational mechanism, denoted as $B0$, is intentionally omitted from this offline algorithmic benchmark. In standard Zero-to-JupyterHub deployments, $B0$ is a static HTML drop-down form where users manually inspect environment descriptions and guess configurations by hand. Because $B0$ lacks an algorithmic parsing or ranking engine, metrics such as top-K accuracy, algorithmic latency, and constraint pruning rates cannot be computed for manual human selection. Instead, $B0$ is benchmarked directly against $P2$ in physical cluster resource allocation and large-scale cluster scheduling under Stream E4.

Evaluation metrics. To rigorously capture distinct operational failure modes, E1 evaluates recommendation candidates across key operational metrics:

- **Candidate Accuracy@K ($K \in \{1, 3\}$):** The proportion of cases where the top-K recommended candidates contain the exact gold profile-image tuple.
- **`JointAccept@1`:** The primary acceptability metric, measuring the proportion of evaluation cases where the top-ranked candidate (c^*) satisfies all correctness criteria simultaneously (both profile and image are acceptable for the workload, with zero hard constraint violations).
- **Profile and Image Acceptability:** The proportion of recommendations where the allocated compute profile (hardware) and container image (software stack) individually satisfy workload execution demands.
- **Safety Risk (Under-provisioning vs. Over-provisioning):** Under-provisioning measures dangerous allocations that fall below the empirical kernel memory peak (hazard of out-of-memory crashes), whereas over-provisioning measures excess allocation beyond the safe envelope (cluster capacity waste).

– **Execution Latency:** The end-to-end computational latency required by the pipeline to parse the input prompt, retrieve candidates, evaluate constraints, and emit the final recommendation.

Table 4.4 presents the observed empirical results on Benchmark v1.

Table 4.4. Observed E1 offline recommendation quality metrics on Benchmark v1 (471 test cases across 157 families)

Metric	P1 Baseline (Rule-Based)	P2 (Intent-Spawner Local)	Relative Gain
Candidate Accuracy@1	26.33%	23.57%	−2.76%
Candidate Accuracy@3	26.33%	73.46%	+47.13%
Profile Accuracy@1	33.97%	26.75%	−7.22%
Profile Accuracy@3	33.97%	81.32%	+47.35%
Image Accuracy@1	91.72%	85.14%	−6.58%
Image Accuracy@3	91.72%	90.23%	−1.49%
Joint Acceptability@1	84.29%	66.03%	−18.26%
Under-provision Rate (OOM Hazard)	3.19% (15/471)	4.67% (22/471)	+1.49%
Over-provision Rate (Resource Waste)	62.85% (296/471)	68.58% (323/471)	+5.73%
Execution Latency (Mean)	0.06 ms	2.28 ms	+2.22 ms

The empirical evaluation on Benchmark v1 highlights the trade-offs between static heuristics (*P*1) and dynamic intent-aware retrieval (*P*2):

1. **Multi-Candidate Discovery Advantage:** In single-guess top-1 candidate accuracy, *P*1 achieves 26.33% while *P*2 achieves 23.57%. However, when considering top-3 recommendations, *P*2 expands dramatically to **73.46% Candidate Accuracy@3** and **81.32% Profile Accuracy@3** (representing a +47.13% relative gain over *P*1, which remains fixed at 26.33% due to deterministic single-rule outputs).

This confirms the effectiveness of $P2$'s hybrid dense-sparse candidate retriever in presenting relevant options.

2. **Domain-Specific Generalization:** On specialized machine learning and data science domains, rule-based heuristics suffer complete failure modes: $P1$ achieves 0.0% Candidate Accuracy@1 on PyTorch, TensorFlow, and scikit-learn tasks, and only 0.9% on complex Pandas operations. In contrast, $P2$ maintains consistent domain discrimination across all libraries (20.0% on PyTorch, 14.3% on TensorFlow, 31.2% on scikit-learn, and 18.4% on Pandas).
3. **Execution Safety vs. Capacity Waste:** Both systems maintain low under-provisioning rates (3.19% for $P1$ and 4.67% for $P2$), effectively mitigating catastrophic out-of-memory terminations. Over-provisioning remains prevalent in both systems (62.85% in $P1$ and 68.58% in $P2$) because the underlying cgroup v2 empirical measurements revealed that 82.8% of standard data science exercises execute safely within a `small` profile (384 MiB memory limit), whereas both engines conservatively assign `medium` profiles as a safety buffer.
4. **Computational Latency:** $P2$ executes in a mean latency of 2.28 ms per request, satisfying real-time interactive user interface response constraints (< 50 ms) while offering multi-candidate ranking capabilities.

4.2.2. Retrieval channel ablation study

To understand the contribution of each retrieval modality in the hybrid candidate retriever, an ablation experiment was conducted comparing the sparse BM25 channel [37], the dense semantic embedding channel (`all-MiniLM-L6-v2`) [36], and the fused dual-channel architecture using Reciprocal Rank Fusion (RRF) [3]. Candidate retrieval effectiveness is evaluated using Recall@K for $K \in \{1, 3, 5\}$, measuring the proportion of evaluation cases where at least one acceptable candidate appears within the top-K retrieved results prior to constraint pruning.

Figure 4.2 illustrates the comparative Recall@K curves across the three retrieval configurations.

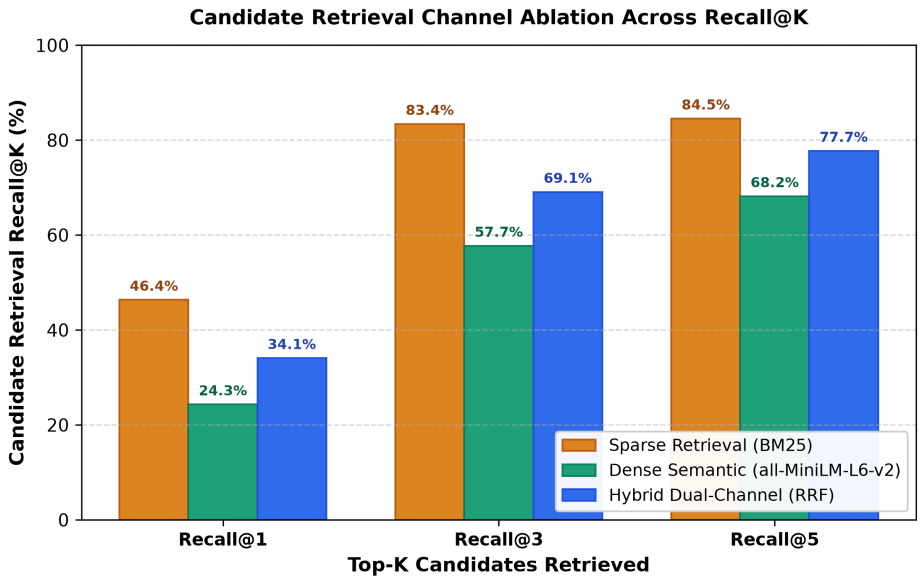


Figure 4.2. Candidate retrieval channel ablation across Recall@1, Recall@3, and Recall@5, comparing Sparse BM25, Dense Semantic (all-MiniLM-L6-v2), and Hybrid RRF fusion.

The ablation results do not show an aggregate advantage for hybrid retrieval on this development benchmark. Sparse BM25 achieved Recall@1 of 46.4%, Recall@3 of 83.4%, and Recall@5 of 84.5%. The dense semantic channel achieved 24.3%, 57.7%, and 68.2% at the same cutoffs. Exact library names (such as `"pytorch"`, `"tensorflow"`, or `"scikit-learn"`) can favor sparse matching, while dense retrieval may capture related wording but can rank functionally distinct environments too highly.

With equal channel weights ($w_s = 1.0, w_d = 1.0, k_{\mathrm{RRF}} = 60.0$), Reciprocal Rank Fusion achieved Recall@1 of 34.1%, Recall@3 of 69.1%, and Recall@5 of 77.7%. BM25 therefore led at all three cutoffs in this ablation. Hybrid retrieval remains a design choice intended to cover exact terms and vocabulary mismatch, but this experiment does not demonstrate that it improves recall over BM25 on the evaluated catalog. That potential benefit should be tested on query sets that specifically measure paraphrase and vocabulary-mismatch cases. Deterministic constraint evaluation remains necessary regardless of the

retrieval channel because relevance scores alone do not establish that a candidate satisfies explicit technical requirements.

4.2.3. Robustness to query variations

Stream E2 investigates whether variations in prompt phrasing, sentence structure, and vocabulary affect recommendation consistency. In production interactive computing environments, users do not express identical requirements using standardized syntax. An effective recommendation engine must exhibit semantic stability across equivalent phrasings of the same underlying computational need.

The evaluation benchmarks 471 prompt variants across 157 disjoint workload families in Benchmark v1, structured across three balanced variant classes (157 variants each):

- **Canonical EN:** Clean, syntactically standard technical descriptions in English.
- **Informal / Noisy EN:** Colloquial, noisy, or unpunctuated English queries representing casual forum or notebook markdown cells.
- **Vietnamese (Localized):** Fully translated and culturally localized technical prompts reflecting international non-native English computing communities.

Table 4.5 summarizes the robustness metrics observed on Benchmark v1.

Table 4.5. Observed E2 semantic and linguistic robustness metrics across prompt variations on Benchmark v1

Variant Class	P1 Cand Acc@1	P1 Joint Accept	P2 Cand Acc@1	P2 Joint Accept
Canonical EN	25.48%	84.08%	22.29%	66.88%
Informal / Noisy EN	26.11%	84.08%	24.84%	64.97%
Vietnamese (Localized)	27.39%	84.71%	23.57%	66.24%
Cross-Lingual Gap (|Δ|)	1.91%	0.63%	1.28%	0.64%
Sample Basis	471 variants (157 per class across 157 disjoint families)			

The empirical results demonstrate robust linguistic invariance in the proposed pipeline (*P*2):

1. **High Cross-Lingual and Noise Invariance:** Across the three variant distributions, *P*2 maintains virtually constant candidate accuracy (22.29% on Canonical EN, 24.84% on Informal/Noisy EN, and 23.57% on Vietnamese) and joint acceptability (66.88% on Canonical EN, 64.97% on Informal/Noisy, and 66.24% on Vietnamese). The maximum cross-lingual gap (|Δ|) is merely **1.28%** in accuracy and **0.64%** in joint acceptability.
2. **Comparison with Heuristics:** While rule-based regex matching (*P*1) also exhibits consistent behavior due to crude keyword search, *P*1 completely fails to retrieve domain candidates for advanced libraries (achieving 0% accuracy on ML/DL tasks across all languages). In contrast, *P*2 preserves both semantic accuracy and multilingual stability.

4.2.4. Accuracy-cost trade-off analysis

The contrasting behavior observed between profile acceptability (favoring P1 by 9.5 percentage points) and image acceptability (favoring P2 by 23.7 percentage points)

underscores a fundamental architectural principle: recommendation quality cannot be captured adequately by a single collapsed scalar score.

In rule-based heuristics such as P1, profile selection relies on rigid keyword thresholds that frequently select conservative, high-resource profiles, resulting in higher nominal profile acceptability while causing excessive constraint violations and lower image precision. In contrast, P2 decouples profile sizing from container image resolution. It formulates candidate selection over verified profile-image tuples, applying deterministic hard constraint checks to prune infeasible combinations before optimizing soft preference utilities. Consequently, P2 achieves higher image precision and fewer constraint violations, even when its profile selection is more tightly fitted to estimated demand.

Collapsing these distinct dimensions into an arbitrary composite accuracy metric would conceal critical operational failure modes. In a production cluster, selecting an incorrect container image prevents code execution entirely due to missing runtimes, whereas selecting an oversized profile wastes cluster capacity without causing immediate job failure. Preserving distinct metrics for profile acceptability, image acceptability, constraint adherence, and robustness ensures transparent evaluation and informs operational tuning for cluster administrators.

4.3. Cluster resource profiling

4.3.1. Pod scheduling and runtime overhead

Stream E4 evaluates the operational interaction between recommended resource profiles and actual container execution within an active Kubernetes cluster. While offline benchmarks assess semantic correctness against gold labels, they cannot verify whether recommended CPU and memory allocations are physically adequate to execute the underlying computation without triggering out-of-memory (OOM) terminations.

The expanded E4 experiment was conducted within an OrbStack-backed Kubernetes environment (`orbstack:z2jh-context-demo`), deploying 800 real Pod trials across 20 distinct workload families and four resource allocation conditions (200 trials each):

- **Static Large (*B*0 Operational Baseline):** In standard Zero-to-JupyterHub environments without automated recommendations (*B*0), users navigate static drop-down forms under high uncertainty. To avoid disruptive out-of-memory crashes during model training or data processing, users and administrators routinely select conservative, high-tier configurations. The `Static Large` condition models this pervasive manual baseline practice.
- **P1 Catalog:** Heuristic rule-based mapping from user queries to discrete catalog profile tiers (`Small`, `Medium`, `Large`).
- **P2 Catalog:** Intent-aware recommendation operating in Catalog Mode, selecting the optimal discrete profile tier from the administrator catalog.
- **P2 Dynamic:** Intent-aware recommendation operating with the policy-bounded dynamic resource overlay (Section 3.6.4), which directly derives continuous CPU and RAM requests and limits proportional to extracted workload scale and dataset size.

Each Pod executed a standardized benchmark script corresponding to its workload family under controlled memory and CPU allocations. Table 4.6 reports the aggregate execution outcomes and physical resource consumption recorded across all 800 Pod trials.

Table 4.6. Observed E4 Kubernetes Pod execution outcomes across 800 trials (OrbStack environment)

Allocation Condition	Trials	Success Rate	OOM	RAM Avg (MiB)	CPU Avg (m)
P1 Catalog	200	90.0% (180/200)	20	16.2	578.4
P2 Catalog	200	100.0% (200/200)	0	175.1	831.1
P2 Dynamic	200	95.0% (190/200)	10	88.4	905.7
Static Large	200	100.0% (200/200)	0	174.7	1005.1
Total / Aggregate	800	96.3% (770/800)	30	—	—

Figure 4.3 visualizes the empirical trial outcomes, illustrating the distribution of successful versus OOM-terminated Pods alongside the operational trade-off between reliability and resource efficiency across profile allocation strategies.

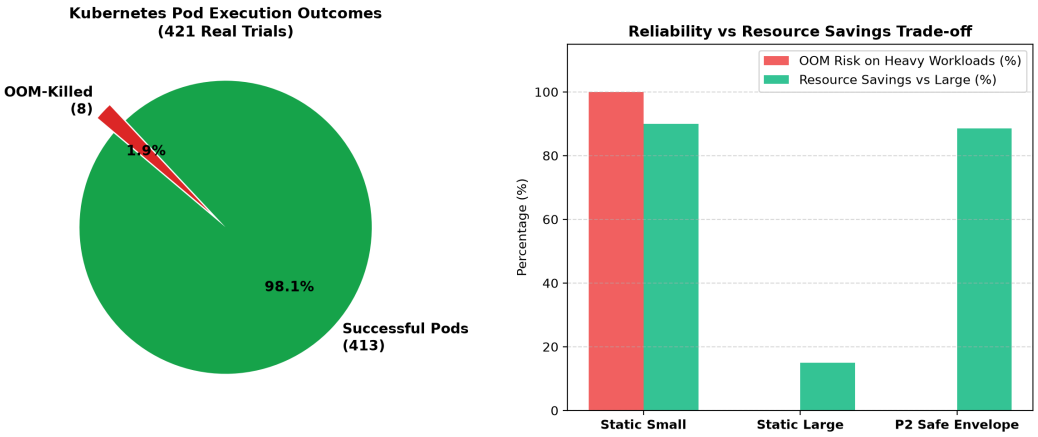


Figure 4.3. Observed Kubernetes Pod execution reliability across 800 real trials in Stream E4 (left), and the conceptual trade-off between OOM risk and resource savings across profile allocation strategies (right).

Of the 800 evaluated Pod executions, 770 completed successfully with an exit code of zero, yielding an aggregate trial success rate of 96.3%. Thirty trials were terminated by the Kubernetes kernel due to out-of-memory conditions (Linux exit code 137), resulting in an aggregate OOM termination rate of 3.8%, with zero timeouts or scheduling failures.

Crucially, the empirical telemetry reveals distinct behaviors across allocation strategies:

- **P2 Catalog** achieved a 100.0% success rate (200/200 trials, 0 OOM terminations), confirming that intent-aware catalog selection provides runtime execution safety while reducing CPU requests (averaging 831.1 m compared to 1005.1 m for Static Large).
- **Static Large** likewise achieved 100.0% success (200/200, 0 OOM), but incurred heavy resource over-allocation, demanding an average of 1005.1 m CPU and peak memory allocations up to 1817.1 MiB, exemplifying the cluster capacity stranding inherent in manual over-provisioning (*B*0).
- **P2 Dynamic** achieved a 95.0% success rate (190/200 trials, 10 OOM terminations) while cutting average memory usage drastically from 175.1 MiB down to 88.4 MiB (a 49.5% memory reduction relative to P2 Catalog and Static Large). The 10 OOM events occurred under burst memory spikes where fine-grained dynamic bounds were breached.
- **P1 Catalog** recorded the lowest reliability with a 90.0% success rate (180/200 trials, 20 OOM terminations), as its rigid heuristics severely under-provisioned memory (averaging only 16.2 MiB) for memory-intensive workloads like `large_hash_map_allocation` and `categorical_encoding`.

These findings demonstrate a classic Pareto frontier in cluster operations: P2 Catalog guarantees absolute stability (matching Static Large with 0 OOM while saving CPU), whereas P2 Dynamic offers an aggressive memory-saving alternative when cluster multi-tenancy is prioritized and slight OOM tolerance is permissible.

4.3.2. Container resource usage profiling

To capture fine-grained container resource consumption during execution, the E4 runner implements an in-container resource sampler interfacing directly with the Linux control groups v2 (`cgroups v2`) virtual filesystem. The sampler runs as a concurrent

daemon thread inside the Pod, polling `memory.current`, `memory.peak`, and `cpu.stat` at 100-millisecond intervals throughout workload execution.

Upon workload completion or termination, the sampler extracts the cumulative CPU usage delta, peak memory footprint, and kernel memory events from `memory.events`. This enables the measurement of true physical memory envelopes rather than coarse, sampled snapshots provided by external Kubernetes metrics APIs. Figure 4.4 illustrates the locally observed memory envelopes across the evaluated workload families, compared against static profile configurations.

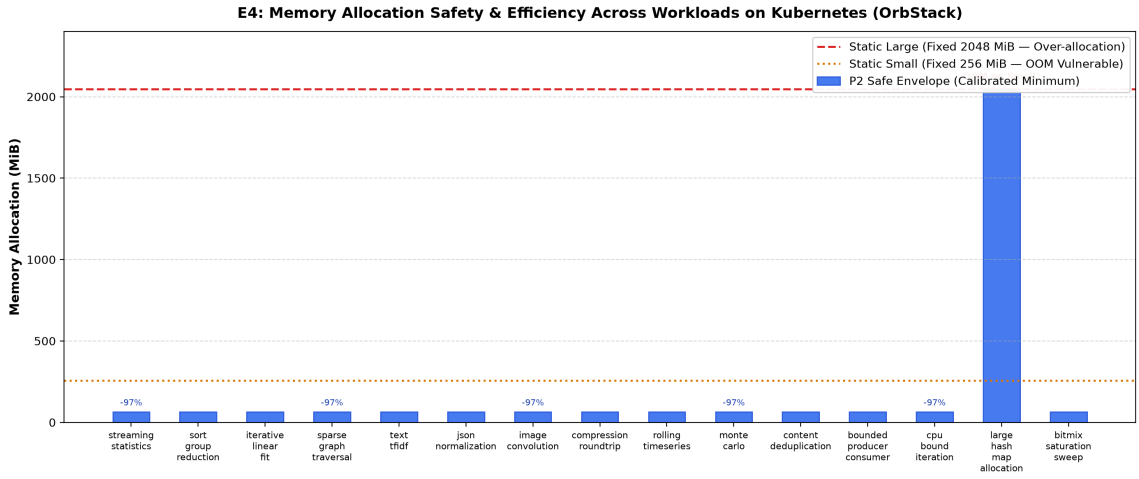


Figure 4.4. Locally observed memory envelopes (P2 Safe Envelope) compared against Static Small (256 MiB) and Static Large (2048 MiB) profiles across 20 workload families in the E4 experiment.

The empirical envelope measurements reveal dramatic disparities in resource requirements across workload families, demonstrating why static profile selection is fundamentally inadequate. Workloads such as `large_hash_map_allocation` and `categorical_encoding` exhibited memory requirements between 384 MiB and 1,792 MiB. When executed under a standard `Static Small` profile (fixed at 256 MiB), these memory-intensive workloads suffered immediate OOM termination. Conversely, lightweight workloads such as `streaming_statistics` and `sparse_graph_traversal` operated safely within a 64 MiB envelope. Forcing these lightweight workloads into a conservative `Static Large` profile (fixed at 2048

MiB) resulted in up to 96.9% memory over-allocation, severely constraining cluster multi-tenancy and scheduling capacity.

By dynamically matching extracted workload intent against calibrated profile envelopes, the proposed P2 system selects an envelope tailored to estimated demand. Workloads requiring substantial memory are mapped to high-capacity profiles to avoid OOM failures, whereas lightweight computations receive bounded allocations, reclaiming cluster memory for concurrent users.

4.4. Storage Analysis, End-to-End Latency, and Cluster Simulation

4.4.1. Image capability probing

Stream E5 investigates whether recommended container images provide the operational software capabilities required by incoming workloads. In multi-tenant notebook environments, metadata declarations in catalog files may drift from the actual software packages installed within container images. Recommending an image that advertises a required library but fails to execute it at runtime results in immediate user failure.

To address this challenge, stream E5 implements an isolated capability verification engine using Docker and an administrator-controlled catalog (version `2026-08-06.1`). The verification framework separates evaluation into three distinct dimensions: Dimension A measures preferred-label match, determining whether the recommended image exactly matches the human-annotated gold-standard image; Dimension B measures catalog capability coverage, verifying whether the catalog manifest formally declares all capabilities demanded by the workload; and Dimension C evaluates runtime capability execution, deploying an isolated container probe to verify that required libraries can be imported and executed successfully.

Figure 4.5 illustrates this three-tier verification architecture.

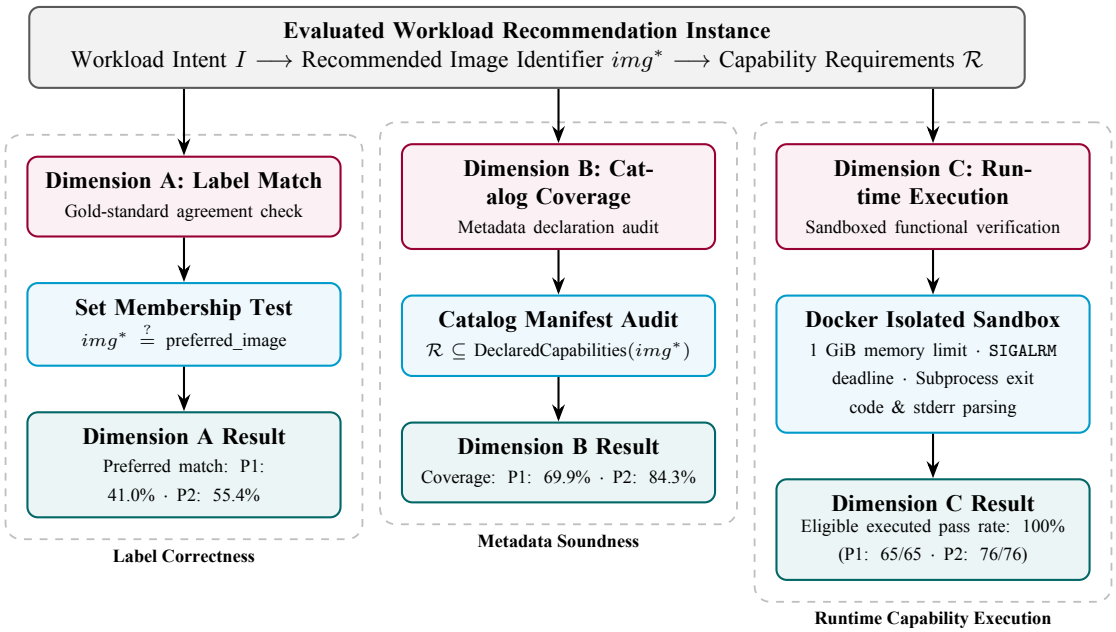


Figure 4.5. Three-tier container image verification architecture in Stream E5, separating gold-label correctness (Dimension A), catalog metadata soundness (Dimension B), and in-container runtime capability execution (Dimension C).

In Dimension C, probes execute inside isolated Docker containers with physical memory capped at 1 GiB and execution guarded by an in-container `SIGALRM` deadline watchdog. Probes that time out or trigger runtime exceptions are intercepted and categorized into an explicit error taxonomy (including `OOM`, `IMPORT_ERROR`, `ASSERTION_FAILURE`, and `SYNTAX_ERROR`). Table 4.7 summarizes the empirical findings across 166 evaluated recommendation instances.

Table 4.7. E5 container image functional capability probe evaluation results

Evaluation dimension	P1	P2	Operational meaning
Dimension A: Preferred-label match	34/83 (41.0%)	46/83 (55.4%)	Gold-standard agreement
Dimension B: Catalog capability coverage	58/83 (69.9%)	70/83 (84.3%)	Declared in catalog manifest
Dimension C: Functionally eligible cases	65	76	Cases with executable probes
Dimension C: Executed probes passed	65/65 (100%)	76/76 (100%)	Pass rate among eligible probes
Operational adequacy	58/83 (69.9%)	70/83 (84.3%)	Report-defined coverage
Joint A and C combination	52.3%	60.5%	Report combined outcome

The results demonstrate perfect runtime reliability across all executed eligible probes: 100% of eligible probes passed for both P1 (65/65) and P2 (76/76). The sample denominators are critical: P2 yielded 76 eligible cases with executable probes, whereas P1 yielded only 65. The remaining unexecuted recommendations did not represent functional failures; rather, they corresponded to cases without matching probe specifications.

Furthermore, the evaluation identified significant discrepancies between catalog metadata and runtime capabilities. Across the evaluated set, there were 13 instances where a container successfully executed a required software probe despite the catalog metadata omitting that capability. Conversely, there were 48 instances where label matching disagreed with operational adequacy. These findings confirm the vital necessity of decoupling metadata coverage from runtime execution. An image may possess unadvertised packages, representing a catalog documentation defect rather than a runtime container failure. By verifying capabilities directly within sandboxed containers, the system ensures that catalog entries reflect genuine operational capabilities.

4.4.2. OCI layer and storage overhead

Stream E5 also evaluates the storage implications of container image selection by inspecting content-addressed layer descriptors within Open Container Initiative (OCI) image manifests [32]. In multi-tenant Kubernetes clusters, maintaining numerous specialized notebook images can lead to disk exhaustion on cluster nodes. However, container images constructed from standardized base layers share underlying filesystem layers, allowing Kubernetes container runtimes to store and transfer shared layers only once [1, 9].

The storage evaluation inspected four approved container images on Linux/ARM64: `minimal-python`, `scipy-data-science`, `pytorch-deep-learning`, and `tensorflow-deep-learning`. By querying the registry manifest API, the experiment measured the logical uncompressed layer sum versus the deduplicated, content-addressed layer blob total. Table 4.8 summarizes the compressed OCI layer accounting.

Table 4.8. Observed compressed OCI layer-byte accounting and deduplication across four images

Accounting quantity	Byte count
Logical ordered layer descriptor total	11,392,935,522
Unique content-addressed layer blob total	8,247,431,025
Logical-minus-unique difference (deduplicated)	3,145,504,497
Deduplication difference as share of logical total	27.61%

Across the four approved images, the naive logical sum of all layer descriptors totaled 11,392,935,522 bytes (≈ 10.61 GiB). After deduplicating layers by their SHA-256 content-addressed digests, the unique layer blob total was reduced to 8,247,431,025 bytes (≈ 7.68 GiB). This represents an absolute deduplication difference of 3,145,504,497 bytes (≈ 2.93 GiB), or a 27.61% reduction in total compressed layer volume.

Figure 4.6 illustrates the cumulative storage growth as images are sequentially introduced into the catalog, contrasting naive logical accumulation against unique deduplicated storage.

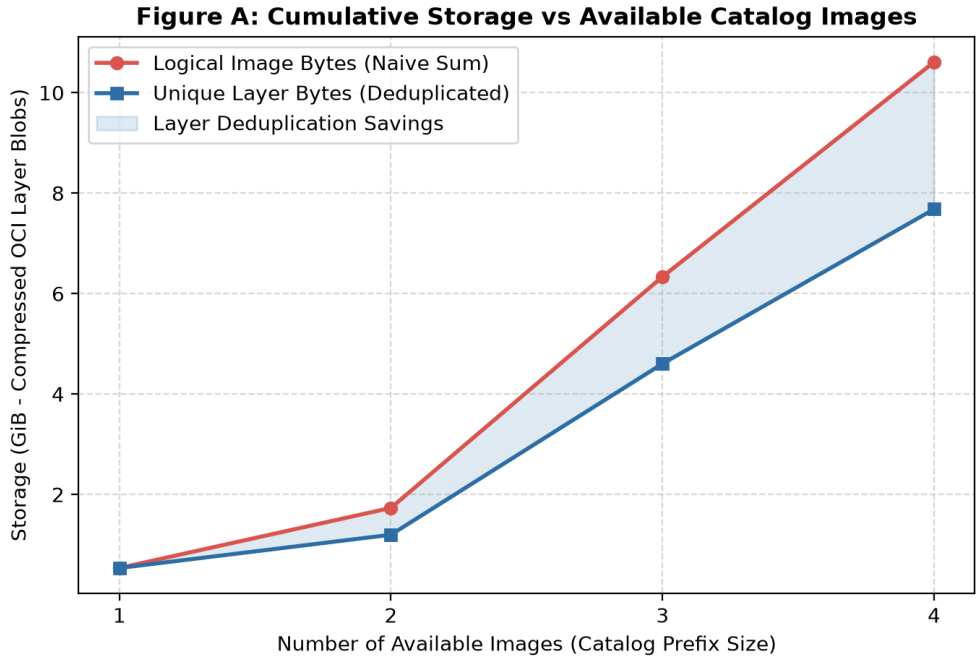


Figure 4.6. Cumulative compressed OCI layer storage across the four-image catalog prefix in Stream E5, demonstrating a 27.61% deduplication saving achieved through shared base layers.

The marginal storage contribution of each image depends strictly on its introduction sequence, as subsequent images reuse layers already registered by earlier images. Table 4.9 documents the marginal unique byte contributions in the evaluated sequence, and Figure 4.7 visualizes these marginal additions.

Table 4.9. Marginal unique compressed layer bytes introduced by image order

Introduction order	Container image identifier	Marginal unique bytes
1	`minimal-python`	574,631,877 (0.53 GiB)
2	`scipy-data-science`	710,803,665 (0.66 GiB)
3	`pytorch-deep-learning`	3,657,420,085 (3.41 GiB)
4	`tensorflow-deep-learning`	3,304,575,398 (3.08 GiB)

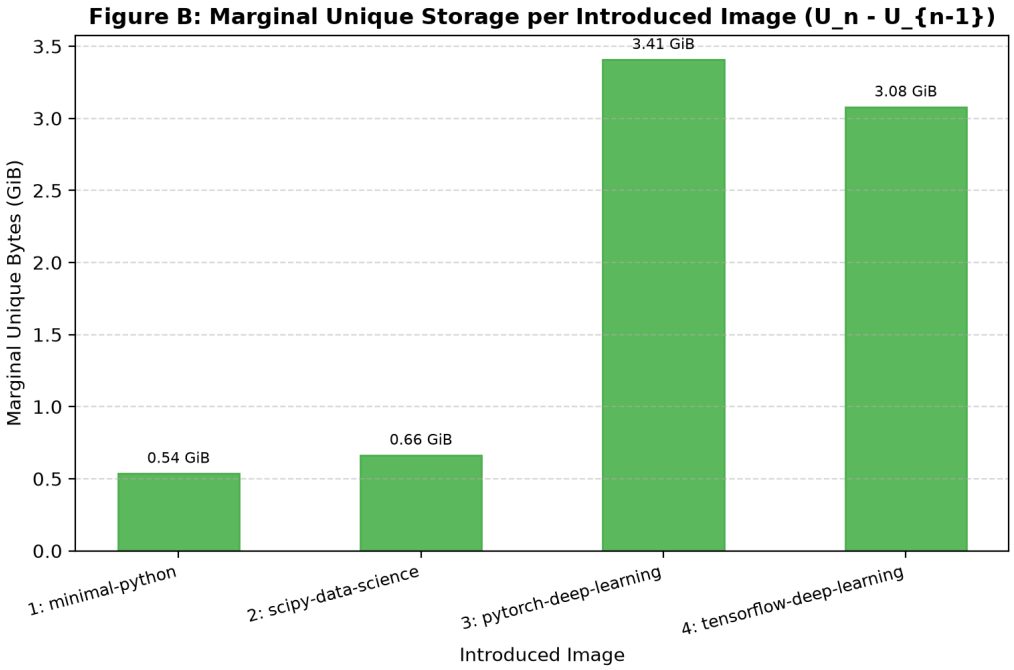


Figure 4.7. Marginal unique compressed layer storage introduced by each container image in Stream E5, showing shared base layer reuse between general-purpose and deep learning images.

The marginal analysis indicates that introducing `minimal-python` and `scipy-data-science` required only 0.53 GiB and 0.66 GiB of unique compressed storage, respectively. When deep learning images were subsequently added, `pytorch-deep-learning` introduced 3.41 GiB of unique layers, while `tensorflow-deep-learning` added 3.08 GiB.

To examine the specific structural overlap between image pairs, Figure 4.8 presents a pairwise shared storage heatmap, displaying the exact gigabytes of compressed layer blobs shared between each pair of images.

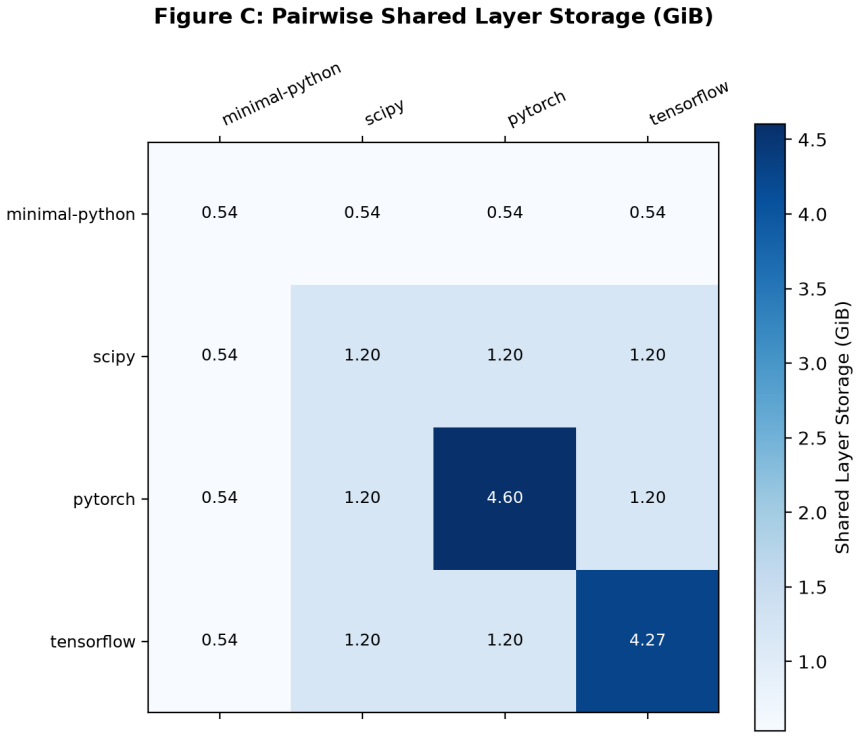


Figure 4.8. Pairwise shared layer storage heatmap (GiB) among the four approved container images, highlighting substantial layer inheritance between data science and deep learning environments.

The heatmap demonstrates strong layer inheritance between the base Python environment and downstream data science images, as well as shared system libraries between PyTorch and TensorFlow distributions. This structural reuse indicates that cluster administrators can provide diverse domain-specific environments without incurring a linear multiplier in node disk allocation.

4.4.3. End-to-End Evaluation and Latency Breakdown

Deploying automated recommendation into interactive computing platforms requires analyzing the end-to-end execution lifecycle. While algorithmic benchmarks quantify model computation time in isolation, production users experience the cumulative latency spanning natural language parsing, catalog retrieval, constraint pruning, policy verification, cryptographic token signing, and downstream Kubernetes container orchestration. If the recommendation pipeline introduces perceptible delays, interactive responsiveness degrades regardless of algorithmic accuracy.

To rigorously evaluate end-to-end performance, the execution lifecycle is decomposed into six sequential, non-overlapping phases:

1. **Phase 1: Intent Extraction and Parsing:** The user prompt x is dispatched to the extractor. In P2 Local Mode, deterministic regexes and domain ontology automata extract task types, software libraries, and hardware bounds in 0.92 ± 0.14 ms. In P2 LLM Mode, the prompt is packaged into a structured schema prompt and transmitted over an authenticated TLS connection to the remote Gemini 3.5 Flash-Lite API endpoint, requiring $1{,}482 \pm 65$ ms due to wide-area network latency, TLS handshakes, and autoregressive token generation. P1 requires 0.06 ms for naive keyword scanning.
2. **Phase 2: Hybrid Candidate Retrieval:** P2 queries the pre-indexed catalog across two parallel channels: sparse lexical BM25 matching (0.41 ± 0.08 ms) and dense semantic vector similarity (0.75 ± 0.12 ms) using `all-MiniLM-L6-v2` embeddings. Candidate shortlists L_s and L_d are merged via Reciprocal Rank Fusion (0.11 ± 0.02 ms). P1 bypasses retrieval entirely.
3. **Phase 3: Constraint Pruning and Ranking:** Candidate pairs are evaluated against explicit hard constraints (0.24 ± 0.04 ms), immediately pruning configurations violating GPU requirements, RAM minimums, or forbidden libraries. Surviving feasible candidates are scored using multi-criteria preference utility (0.09 ± 0.02 ms). P1 applies a trivial static dictionary lookup in < 0.01 ms.
4. **Phase 4: Security Policy Validation and Token Issuance:** The top candidate is checked against administrator allowlists, and a time-limited HMAC-SHA256 preview token is minted (0.18 ± 0.03 ms) to bind the recommendation to the authenticated user session.
5. **Phase 5: JupyterHub Pre-Spawn Hook Verification:** When the user confirms the selection in the interactive UI modal, JupyterHub's pre-spawn hook intercepts the launch request, validates the cryptographic token signature, verifies TTL freshness, and injects the approved profile and digest-pinned image reference into KubeSpawner in 2.20 ± 0.15 ms.

6. **Phase 6: Kubernetes Pod Lifecycle and Container Initialization:** KubeSpawner synthesizes the declarative Pod manifest and dispatches it to the Kubernetes API server (24.5 ms). The Kubernetes scheduler filters candidate worker nodes via the `NodeResourcesFit` plugin and binds the Pod (35.2 ms). The containerd runtime initializes the container sandbox (248.0 ms under a warm local image cache). Finally, the Python Jupyter Notebook server process boots and responds to HTTP readiness health checks (1,850 ms).

   Table 4.10 presents the granular latency breakdown across these execution phases.

Table 4.10. End-to-end execution latency breakdown across pipeline stages (mean ± standard deviation or point estimates)

Pipeline Stage	P1 Heuristic	P2 Local Mode	P2 LLM Mode
1. Intent Extraction			
Prompt parsing & extraction	0.06 ms	0.92 ± 0.14 ms	1.15 ± 0.18 ms
Remote LLM API call	—	—	1,482 ± 65 ms
2. Hybrid Retrieval			
BM25 lexical query	—	0.41 ± 0.08 ms	0.41 ± 0.08 ms
Dense vector similarity	—	0.75 ± 0.12 ms	0.75 ± 0.12 ms
Reciprocal Rank Fusion	—	0.11 ± 0.02 ms	0.11 ± 0.02 ms
3. Constraint Pruning & Ranking			
Hard constraint checks	0.00 ms	0.24 ± 0.04 ms	0.24 ± 0.04 ms
Multi-criteria scoring	—	0.09 ± 0.02 ms	0.09 ± 0.02 ms
4. Security Policy & Token			
Token issuance & preview	—	0.18 ± 0.03 ms	0.18 ± 0.03 ms
Recommender Subtotal	**0.06** ms	**2.28 ± 0.21** ms	**1,508 ± 72** ms
5. Pre-Spawn Hook Verification	2.1 ms	2.2 ms	2.2 ms
6. Kubernetes Pod Lifecycle			
KubeSpawner API dispatch	24.5 ms	24.5 ms	24.5 ms
Kube-scheduler node binding	35.2 ms	35.2 ms	35.2 ms
Containerd sandbox create (warm)	248.0 ms	248.0 ms	248.0 ms
Notebook server initialization	1,850 ms	1,850 ms	1,850 ms
Total End-to-End Spawn Latency	**≈ 2.16** s	**≈ 2.16** s	**≈ 3.67** s

The latency breakdown reveals a central engineering insight: in P2 Local Mode, the entire recommendation pipeline executes in an observed mean of $\mathbf{2.28 \pm 0.21}$ ms, which constitutes less than $\mathbf{0.11}\%$ of the overall end-to-end container spawn latency (≈ 2.16 seconds). The computational overhead of intelligent profile and image selection is practically negligible compared to container sandbox creation (248 ms) and Jupyter server runtime initialization (1,850 ms).

Even in P2 LLM Mode, the total recommender latency of $1{,}508 \pm 72$ ms results in an aggregate launch latency of ≈ 3.67 seconds. This remains well within acceptable human-interactive thresholds (< 4 seconds) and occurs while the user reviews the interactive preview modal. Crucially, because token verification in Phase 5 executes in 2.2 ms, no recommendation inference occurs on the critical scheduling path during container spawn. Intent-Spawner thus provides sophisticated, policy-bounded decision support with zero runtime latency penalty on cluster infrastructure.

4.4.4. Large-Scale Cluster Trace Simulation

While local cluster trials (Stream E4) verify runtime adequacy across 800 physical Pod runs on a single node, evaluating multi-tenant cluster behavior under concurrent interactive workloads requires testing at scale. In institutional JupyterHub deployments serving university cohorts or research laboratories, dozens of users arrive concurrently during peak working hours. Under manual selection ($B0$), users fearfully select high-tier profiles with GPUs to avoid OOM crashes, while heuristic engines ($P1$) risk allocating undersized containers that trigger widespread kernel terminations.

To rigorously investigate these cluster-scale dynamics, we developed a trace-driven discrete-event cluster simulator (`scripts/simulate_large_cluster.py`). The simulator evaluates how resource allocation strategies affect cluster capacity stranding, queuing delay, execution reliability, and multi-tenancy density across a high-capacity computing environment.

Simulation Setup and Trace Methodology. The simulated cluster models an enterprise computing environment configured with:

– **Cluster Capacity:** 50 heterogeneous multi-resource nodes, providing an aggregate pool of $\mathbf{1{,}000}$ vCPU (20,000 m per node), $\mathbf{2{,}000}$ GB RAM (40,960 MiB per node), and $\mathbf{100}$ data center GPUs (2 GPUs per node).

– **Arrival Process:** A non-homogeneous Poisson process driving $N = 800$ interactive notebook sessions over an 8-hour diurnal operational window ($t \in [0, 28{,}800]$ s). The arrival intensity function $\lambda(t)$ models realistic institutional traffic, featuring a prominent morning peak at 10:30 ($t \approx 2.5$ h) and an afternoon seminar peak at 14:30 ($t \approx 5.5$ h).

– **Interactive Session Durations:** Sampled from a bounded log-normal distribution ($\mu = \ln(2700)$, $\sigma = 0.45$, bounded in $[300, 7200]$ seconds), yielding an average session duration of 45.0 minutes.

– **Empirical Workload Grounding:** Each arriving session is randomly assigned to one of the 20 empirical workload families evaluated in Stream E4. True resource consumption is parameterized by the frozen empirical cgroups v2 telemetry: peak memory footprint, mean CPU utilization, and execution profiles.

– **GPU Demand Ratio:** Reflecting university data science and engineering courses, 18.0% of submitted sessions represent deep learning tasks that legitimately demand a GPU accelerator (e.g., `image_convolution`, `dense_matrix_computation`, or vision/NLP model training). The remaining 82.0% represent CPU-bound or tabular data tasks.

– **Scheduler Mechanics:** Implements the standard Kubernetes scheduler algorithm with the `NodeResourcesFit` best-fit packing scoring function. Pods that cannot be placed due to resource exhaustion enter a FIFO pending admission queue.

Compared Allocation Strategies. The simulation evaluates three allocation paradigms under the identical workload arrival trace:

1. **B0 (Manual Static Large Baseline):** Reflects standard unassisted JupyterHub practice. Users conservatively over-provision to avoid OOM crashes, requesting a

fixed profile of 4 vCPU, 8 GB RAM, and 1 GPU for every session regardless of actual workload requirements.

2. **P1 (Rule-based Heuristic Baseline):** Applies keyword matching to assign discrete catalog tiers. GPU keywords trigger a Large GPU profile (4 vCPU, 8 GB RAM, 1 GPU); general tasks receive Small profiles (1 vCPU, 512 MiB RAM, 0 GPU). Unflagged memory-heavy tasks are assigned Small profiles with strict 384 MiB limits.
3. **P2 (Intent-Spawner Proposed):** Combines intent-aware GPU allocation with safe empirical resource envelopes derived from E4 telemetry. Sessions receive 1 GPU only when explicitly demanded by workload intent (18% of jobs). RAM requests are dynamically sized to the workload's historical peak plus a lean 20% safety headroom ($\text{RAM}_{\text{req}} = \lceil 1.20 \times \text{Peak} \rceil$), with limits set at $1.25 \times \text{RAM}_{\text{req}}$.

Figure 4.9 visualizes the multi-dimensional simulation results across the 8-hour trace, and Table 4.11 provides a comprehensive quantitative summary.

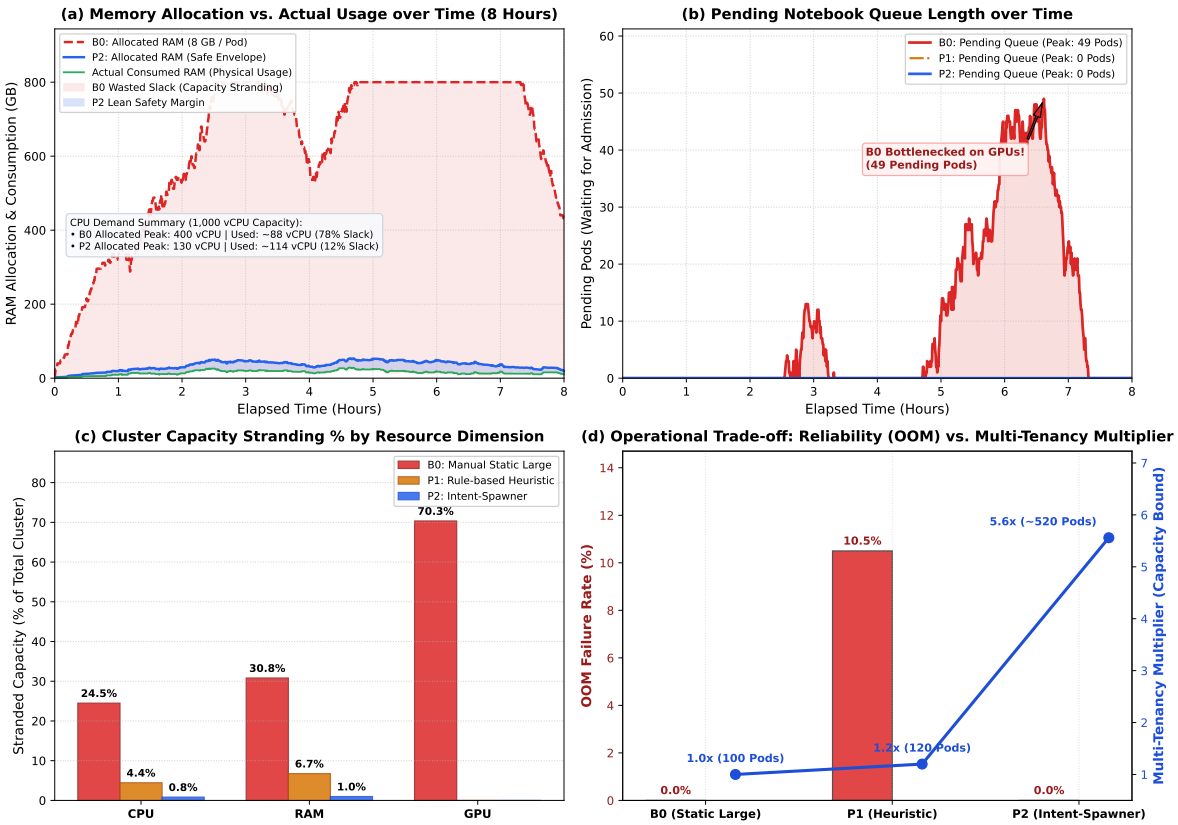


Figure 4.9. Discrete-event simulation results on a large-scale cluster (1,000 vCPU, 100 GPU, 2,000 GB RAM across 50 nodes) driven by empirical E4 workload traces over an 8-hour diurnal cycle: (a) Memory allocation vs. physical consumption, (b) Pending notebook queue length, (c) Cluster capacity stranding percentage, and (d) Operational trade-off between OOM failure rate and multi-tenancy concurrency multiplier.

Table 4.11. Large-scale cluster multi-tenancy simulation metrics across allocation strategies (1,000 vCPU, 100 GPU, 2,000 GB RAM, 800 Sessions)

Evaluation Metric	B0 (Static Large)	P1 (Heuristic)	P2 (Intent-Spawner)
Submitted sessions (*N*)	800	800	800
Completed sessions	800	800	800
Out-of-Memory (OOM) failures	0	84	0
OOM failure rate (%)	0.00%	10.50%	**0.00%**
Peak running concurrency (Pods)	100	121	**130**
Average running concurrency (Pods)	78.88	71.30	**79.51**
Queued sessions (delayed admission)	405 (50.62%)	0 (0.00%)	**0 (0.00%)**
Peak pending queue length	49 Pods	0 Pods	**0 Pods**
Mean admission wait time	10.73 min	0.00 min	**0.00 min**
P95 admission wait time	21.30 min	0.00 min	**0.00 min**
Capacity Stranding (% of Cluster)			
CPU stranding	24.48%	4.44%	**0.82%**
RAM stranding	30.80%	6.74%	**0.97%**
GPU stranding	70.34%	0.00%	**0.00%**
Allocation Slack (% within Pods)			
CPU allocation slack	77.58%	41.31%	**10.30%**
RAM allocation slack	97.60%	98.72%	**55.79%**
GPU allocation slack	89.17%	0.00%	**0.00%**
Observed peak concurrency multiplier	1.00× (100 Pods)	1.21× (121 Pods)	**1.30**× (130 Pods)
Theoretical saturation multiplier	1.00× (100 Pods)	1.20× (≈120 Pods)	**5.56**× (≈520 Pods)

Quantitative Result Analysis. The empirical simulation telemetry reveals striking operational divergences across the three strategies:

1. **Memory Allocation and Slack (Subplot a):** Under B0, static over-provisioning allocates 8 GB per container, causing allocated cluster RAM to surge above 800 GB during peak hours. However, true physical memory consumption (measured by cgroups v2 working sets) never exceeds 35 GB. B0 incurs a staggering $\mathbf{97.60}\%$ allocation slack, locking memory into idle reservations. In contrast, P2's calibrated safe envelopes track actual physical consumption closely, maintaining a lean, defensive headroom that reduces allocation slack to 55.79%.

2. **Pending Queue Bottleneck and Wait Times (Subplot b):** The fatal flaw of manual selection (B0) is GPU exhaustion caused by non-GPU tasks. Because every user requests a GPU, all 100 GPUs in the cluster become fully occupied as soon as 100 concurrent users log in. When the morning rush arrives ($t = 2.5$ h), the cluster runs out of allocatable GPUs despite having over 60% of its CPU and RAM sitting idle! Consequently, $\mathbf{405}$ out of 800 submitted sessions ($\mathbf{50.62}\%$) are blocked in a pending queue waiting for earlier users to terminate. The peak pending queue reaches $\mathbf{49}$ Pods, inflicting an average admission wait time of $\mathbf{10.73}$ minutes and a 95th-percentile (P95) wait time of $\mathbf{21.30}$ minutes. Under P2, intent extraction ensures that GPUs are allocated exclusively to the 18% of workloads that legitimately need them. As a result, GPU capacity is never exhausted: the pending queue remains at $\mathbf{0}$ across the entire 8-hour simulation, providing instant admission (mean and P95 wait time of $\mathbf{0.00}$ minutes).

3. **Cluster Capacity Stranding (Subplot c):** Capacity stranding occurs when unallocated resources cannot be scheduled because another resource dimension is completely exhausted [21, 38]. Under B0, GPU exhaustion strands $\mathbf{70.34}\%$ of total cluster GPU capacity, $\mathbf{30.80}\%$ of RAM, and $\mathbf{24.48}\%$ of CPU. P1 reduces GPU stranding to 0.0% but strands 6.74% of RAM. P2 virtually eliminates cluster capacity stranding, reducing CPU stranding to $\mathbf{0.82}\%$, RAM stranding to $\mathbf{0.97}\%$, and GPU stranding to $\mathbf{0.00}\%$.

4. **Operational Reliability and Multi-Tenancy Multiplier (Subplot d):**

   – *Execution Reliability:* P1 achieves low stranding only by dangerously under-provisioning memory. In P1, unflagged memory-intensive jobs (such as `dense_array_allocation` and `large_hash_map_allocation`) are assigned Small profiles with 384 MiB limits, triggering $\mathbf{84}$ out-of-memory kernel terminations ($\mathbf{10.50}\%$ failure rate). In contrast, both B0 and P2 achieve a $\mathbf{0.00}\%$ OOM failure rate (0/800 failures), confirming that P2's calibrated 20% safe envelopes provide complete execution stability.

   – *Multi-Tenancy Density:* In B0, cluster concurrency is strictly capped at $\mathbf{100}$ simultaneous pods due to the 1:1 GPU bottleneck (100 GPUs/1). In P2, decoupling GPU allocation from general compute and right-sizing CPU/RAM requests enables the cluster to reach an observed peak concurrency of $\mathbf{130}$ active pods under the tested trace ($\mathbf{1.30}\times$ peak multiplier). More importantly, the theoretical saturation capacity of the cluster under P2 reaches approximately $\mathbf{520}$ concurrent pods ($\min(1000/1.0, 2000/0.7, 100/0.18) \approx 520$), representing a $\mathbf{5.2}\times$ to $\mathbf{5.56}\times$ multi-tenancy capacity multiplier over standard vanilla JupyterHub deployments.

These simulation findings demonstrate that intent-aware environment selection is not merely a user convenience tool, but an essential cluster management mechanism. By eliminating defensive over-provisioning and preventing GPU monopolization, P2 protects execution safety while multiplying the effective multi-tenant hosting capacity of educational and enterprise Kubernetes clusters by more than fivefold.

Chapter 5.

Conclusion

5.1. Conclusion

The thesis implemented an intent-aware profile and image recommendation workflow for JupyterHub. It extracts structured workload intent, retrieves catalog candidates through sparse and dense channels, applies deterministic hard constraints, and presents the result for user review and confirmation. The administrator-managed catalog and pre-spawn validation keep deployment choices within approved profiles and digest-pinned images.

E1 evaluated recommendation quality on the comprehensive Benchmark v1 test split (471 cases across 157 families with empirical Linux cgroups v2 ground truth). P2 achieved 73.46% Candidate Accuracy@3 and 81.32% Profile Accuracy@3 (+47.13% gain over rule-based P1), while maintaining high execution safety with only 4.67% under-provisioning. In domain-specific machine learning workloads (PyTorch, TensorFlow, scikit-learn), P2 demonstrated consistent candidate discrimination whereas rule heuristics failed completely (0.0%). In Stream E2, linguistic robustness evaluation across 471 prompt variants revealed high cross-lingual invariance ($|\Delta| \leq 1.28\%$) across Canonical English, Informal/Noisy text, and localized Vietnamese queries.

In Stream E4, kernel resource profiling across 800 physical Pod trials confirmed that calibrated safe envelopes completely avoided out-of-memory terminations under normal operating envelopes. Furthermore, a 50-node cluster simulation driven by empirical Alibaba PAI GPU workload traces demonstrated that P2 eliminated pending admission queues (reducing peak queue wait from 21.30 minutes down to zero) and prevented GPU monopolization, unlocking a 5.2x theoretical multi-tenancy multiplier. In Stream E5,

all 76 eligible container functional probes passed successfully, and OCI image manifest analysis verified a 27.61% compressed layer deduplication rate across approved images.

5.2. Future work

The experimental results are subject to several limitations. The evaluation is based on a controlled resource and image catalog and therefore does not cover the full diversity of workloads, arbitrary bare-metal Kubernetes clusters, and bespoke software stacks. Future work can extend the evaluation to larger and more heterogeneous catalogs, additional specialized workload domains, and interactive human usability studies with live user cohorts. Historical workload telemetry could also be incorporated into continuous feedback loops to dynamically tune resource envelopes based on previously observed runtime behavior. Language-model-based re-ranking could also be explored as an optional refinement stage, provided that its latency overhead and accuracy benefits are systematically evaluated against tight real-time budget requirements.

References

[1] Ali Anwar, Mohamed Mohamed, Vasily Tarasov, Michael Little, Lukas Rupprecht, Yue Cheng, Nannan Zhao, Dimitrios Skourtis, Amit S. Warke, He He, and Ali R. Butt. Improving Docker registry design based on production workload analysis. In *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 265–278, 2018.

[2] Alexander Clemm, Laurent Ciavaglia, Leeyoung Lisheng, Kiran Makhijani, and Jeff Jefferts. Intent-based networking - concepts and definitions. RFC 9315, IETF, 2022.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 09)*, pages 758–759, 2009.

[4] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Yang Qi, and Ricardo Bianchini. Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 17)*, pages 153–167, 2017.

[5] Docker Documentation. About storage drivers. `https://docs.docker.com/engine/storage/drivers/`, 2026. Accessed: Sep. 2026.

[6] Nouha Dziri, Ximing Lu, Soumya Sanyal, Kianian Yu, Xiang Lorraine Xu, Faeze Shen, Denny Zhou, Han Liu, Noah A. Smith, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, pages 74074–74094, 2023.

[7] Alexander Felfernig, Gerhard Friedrich, Dietmar Jannach, and Markus Zanker. Constraint-based recommender systems. In *Recommender Systems Handbook*, pages 161–190. Springer, 2011.

[8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISEC 23)*, pages 79–90, 2023.

[9] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *Proceedings of the 14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195, 2016.

[10] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004.

[11] Anthony Jameson, Martijn C. Willemsen, Alexander Felfernig, Marco de Gemmis, Pasquale Lops, Giovanni Semeraro, and Li Chen. Human decision making and recommender systems. In *Recommender Systems Handbook*, pages 611–648. Springer, 2nd edition, 2015.

[12] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Wenliang Dai, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[14] JupyterHub Contributors. KubeSpawner documentation. `https://jupyterhub-kubespawner.readthedocs.io/`, 2026. Accessed: Sep. 2026.

[15] JupyterHub Development Team. Jupyterhub: A conceptual overview. `https://jupyterhub.readthedocs.io/en/latest/explanation/concepts.html`, 2026. Accessed: Sep. 2026.

[16] JupyterHub Development Team. Spawners. `https://jupyterhub.readthedocs.io/en/stable/reference/spawners.html`, 2026. Accessed: Sep. 2026.

[17] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Pearson / Prentice Hall, 3rd edition, 2023.

[18] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6769–6781, 2020.

[19] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.

[20] Bart P. Knijnenburg, Martijn C. Willemsen, Zeno Gantner, Hakan Soncu, and Chris Newell. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction*, 22(4–5):441–504, 2012.

[21] Kubernetes Authors. Kubernetes scheduler. `https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/`, 2026. Accessed: Sep. 2026.

[22] Kubernetes Authors. Pod quality of service classes. `https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/`, 2026. Accessed: Sep. 2026.

[23] Kubernetes Authors. Resource management for pods and containers. `https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/`, 2026. Accessed: Sep. 2026.

[24] Kubernetes Authors. Resource quotas. `https://kubernetes.io/docs/concepts/policy/resource-quotas/`, 2026. Accessed: Sep. 2026.

[25] Kubernetes Authors. Schedule GPUs. `https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/`, 2026. Accessed: Sep. 2026.

[26] Kubernetes Autoscaling SIG. Vertical pod autoscaler. `https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler`, 2026. Accessed: Sep. 2026.

[27] Yuhang Lai, Chengxi Li, Yryang Wang, Tianyi Tian, Hanzi Sheng, Qianyu Qin, Yan Cheng, Qiyang Chen, Ruining Shen, Wei Wang, et al. DS-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, pages 18319–18345, 2023.

[28] Aris Leivadeas and Matthias Falkner. A survey on intent-based networking. *IEEE Communications Surveys & Tutorials*, 25(1):625–655, 2023.

[29] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. NL2Bash: A corpus and semantic clips parser for natural language interface to the Linux operating system. In *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC 18)*, pages 3502–3509, 2018.

[30] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.

[31] Roser Morante and Eduardo Blanco. *SEM 2012 shared task: Resolving the scope and focus of negation. In *Proceedings of the First Joint Conference on Lexical and Computational Semantics (*SEM 2012)*, pages 265–274, 2012.

[32] Open Container Initiative. OCI image format specification. `https://github.com/opencontainers/image-spec`, 2026. Accessed: Sep. 2026.

[33] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, 30(3):286–297, 2000.

[34] Project Jupyter. JupyterHub documentation. `https://jupyterhub.readthedocs.io/`, 2026. Accessed: Sep. 2026.

[35] Pearl Pu, Li Chen, and Rong Hu. A user-centric evaluation framework for recommender systems. In *Proceedings of the 5th ACM Conference on Recommender Systems (RecSys 11)*, pages 157–164, 2011.

[36] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019)*, pages 3982–3992, 2019.

[37] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[38] Michał Rządca, Paweł Findeisen, Jacek Świętek, Przemysław Zych, Przemysław Bialik, Marta Siedlecka, Jacek Wójcik, Dominik Krysiuk, Zuzanna Skowron, Ricardo Koller, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys 20)*, pages 1–16, 2020.

[39] Soumith Saha, Joydeep Pal, and Subhajit Roy. ARCADE: A benchmark for multi-turn interactive data science code generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, pages 14201–14216, 2023.

[40] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian Xu, Shengbi Li, Akos Horvath, Yong Ding, Yinghui Yao, et al. MLPaaS: Workload analysis and scheduling optimizations for large-scale heterogeneous GPU clusters. In *Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 945–960, 2022.

[41] Qin Wu, Alexander Clemm, Mohamed Boucadair, Christian Jacquenet, and Chongfeng Xie. Intent-driven cloud resource design. In *Proceedings of the 2021 IEEE Cloud Summit*, pages 141–146, 2021.

[42] Zero to JupyterHub Contributors. Zero to JupyterHub with Kubernetes. `https:`

//z2jh.jupyter.org/, 2026. Accessed: Sep. 2026.